SkyVoyager: On Using Multi-Region Spot Instances to Minimize AI Batch Job Cost

Abstract

AI batch jobs such as model training, inference pipelines, and data analytics require substantial GPU resources and often need to finish before a deadline. Spot instances offer 3–10× lower cost than on-demand instances, but their unpredictable availability makes meeting deadlines difficult. Existing systems either rely solely on spot instances and risk deadline violations, or operate in simplified single-region settings. These approaches overlook substantial spatial and temporal heterogeneity in spot availability, lifetimes, and prices. We show that exploiting such heterogeneity to access more spot capacity is the key to reduce the job execution cost.

We present `SkyVoyager`, a multi-region scheduling system that maximizes spot usage and minimizes cost while guaranteeing deadlines. `SkyVoyager` uses lightweight probing to estimate availability, predicts spot lifetimes, accounts for migration cost, and unifies regional characteristics and deadline pressure into a monetary cost model that guides scheduling decisions. Our evaluation shows that `SkyVoyager` achieves 1.25–3.96× cost savings in real cloud deployments and performs within 10% cost differences of an optimal policy in simulation, while consistently meeting deadlines.

1 Introduction

The emerging generative AI workloads demand unprecedented computational capacity. Large-scale model training, offline inference, and data analytics all require substantial GPU resources and often take hours or days to complete [10, 12, 21, 28, 53]. These long-running jobs, herein referred to as *AI batch jobs*, often need to be completed by a given *deadline*. Such deadlines arise naturally and are workload-dependent, for example, periodic data processing must complete within a fixed time window (e.g., minutes to hours) to ensure that downstream analytics or dashboards reflect real-time data.

As model sizes and system complexity grow, these jobs become increasingly costly to operate. In this paper, we focus on minimizing the cost of running AI batch jobs while still meeting their deadlines. Many of these jobs assume they are interruptible and must tolerate failures (§2.1), because such jobs often run for long periods, and GPU instances can experience failures at non-negligible rates [17, 46], the likelihood of a failure during execution is significant. To avoid wasting computation when failures occur, these jobs typically implement checkpoint-based recovery [19, 39, 40, 45, 55, 57, 59].

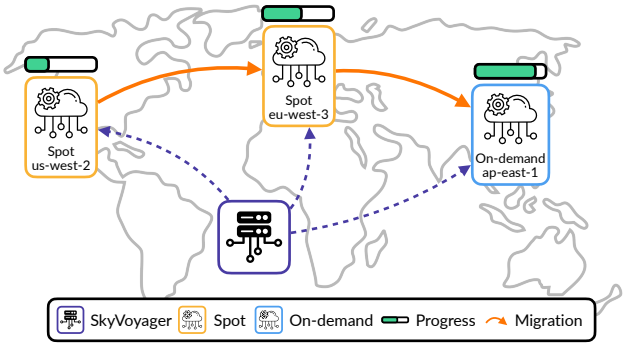


Figure 1: `SkyVoyager` Overview. `SkyVoyager` migrates AI batch jobs across multiple regions to harness available spot instances and minimize the cost. It monitors job progress and switches to on-demand instances when spot capacity is unavailable, ensuring that the job completes before its deadline.

To reduce cost, we can take advantage of a job's *slack time*, which is the amount of delay it can tolerate before risking its deadline, and the interruptible nature of these jobs by using *spot instances*. Cloud providers offer these instances as a cheaper alternative to on-demand instances, typically 3–10× lower in cost [3]. However, they are subject to two limitations: preemption by the cloud provider and temporary unavailability in the spot market. Therefore, spot capacity alone cannot guarantee that deadlines are met and must be used together with on-demand instances. To maximize cost savings while still meeting the deadline, we need to consider *where* and *when* to use spot instances due to the heterogeneity they exhibit, spatially across regions and temporally across

time. When chosen properly, it is possible to access more spot capacity at lower prices and further reduce cost, especially for AI batch jobs that use GPUs and experience frequent preemptions [35]. Existing work (§2.2, [4, 7, 14, 25, 52, 60]) explores this opportunity but does not jointly consider the spatial and temporal dimensions of spot heterogeneity. Next, we describe these two forms of heterogeneity in detail.

Spatial heterogeneity across regions. In public cloud providers, availability of spot instances can differ widely across cloud regions (§2.3.1). For example, `eu-central-1` may still have spot capacity while `us-east-1` has none. By leveraging spot instances across regions, a job can continue running on spot to make progress even when one region runs out of capacity, and benefiting from more cost savings. Pricing strategies also vary based on supply and usage patterns across regions, and can differ by up to 5× [5, 15] (§2.3.3), creating opportunities to proactively migrate to cheaper regions to save costs. However, combining these opportunities is challenging. Region availability is unknown ahead of time, and the scheduler must navigate multi-dimensional tradeoffs when availability and price diverge across regions, such as selecting between a high-cost region with high availability and a low-cost region with limited capacity. In addition, migrating jobs across regions with large checkpoints incurs non-trivial monetary egress cost [1, 13, 37], which also vary by source region, so migration must be justified with improved availability or a lower price [29, 51].

Temporal heterogeneity across time. Spot instance lifetimes, meaning the duration between provisioning and preemption, vary significantly across time (§2.3.2). We observe that short spot lifetimes, caused by frequent preemptions, often occur in short volatile periods. It is beneficial to avoid these periods and let the job use spot instances with longer lifetimes, which amortizes cold start costs (the cost to resume from the last checkpoint) and improves goodput. However, predicting spot lifetimes remains challenging. Meanwhile, spot availability can change significantly over time (§2.3.1) and further complicate the scheduling. Early in the schedule, the policy can afford to wait for future periods with higher availability, running during those periods and pausing when unavailable to increase spot usage. As time passes and the deadline becomes tighter, the policy must become more conservative and prioritize steady progress using the resources available at the moment, trading off the possibility of better availability later against the need to ensure timely completion.

In summary, real cloud environments are complex and exhibit both spatial and temporal heterogeneity. Prior deadline-aware efforts [60] explore this problem but consider only a single region, where many of these challenges do not arise. Extending to multiple regions is fundamentally harder because spatial heterogeneity must be leveraged while accounting for migration cost, and temporal heterogeneity must be evaluated across regions. A scheduler operating under these conditions must decide, at any point in time, whether to (i) use a spot instance in the current region or wait for one to appear, (ii) migrate to another region where spot capacity is available, or (iii) use an always-available on-demand instance to make progress to meet the deadline.

We design a multi-region spot scheduling policy to address these challenges. We formulate the scheduling problem as a stochastic optimal control problem and identify its Bellman structure. This structure motivates a greedy utility rule that yields near-optimal performance when the utility of each option is estimated accurately (§3.2). We further show that this utility reduces to the *monetary value of the remaining progress* (§3.3), adjusted by each region's expected spot lifetime. We estimate spot lifetimes from historical availability information using a virtual-instance view of availability probing, which significantly reduces probing cost (§3.4). The scheduler continuously evaluates all candidate regions and opportunistically migrates when a more cost-effective option appears (§3.5).

We implement this policy in `SkyVoyager` (Figure 1), an AI batch job execution system that draws spot capacity from multiple regions to complete jobs within deadlines. Users specify jobs with periodic checkpointing, and `SkyVoyager` migrates both computation and checkpoints across regions while following the policy's decisions to minimize cost.

To evaluate `SkyVoyager`, we deploy it on public clouds to run real batch jobs and experience real-time preemptions and migrations. As shown in §5.1, `SkyVoyager` achieves 1.25–3.96× cost savings while meeting deadlines, compared to both prior research systems (e.g., Uniform Progress [60]) and production systems (e.g., Amazon SageMaker [4]). We further conduct a comprehensive evaluation by replaying real preemption traces and comparing against existing policies, showing that `SkyVoyager` stays within 10% of an omniscient policy across a wide range of configurations. These results demonstrate that, by leveraging deadline slack and drawing spot capacity from multiple regions, it is feasible to drastically reduce the cost of AI batch jobs.

In summary, this paper makes four contributions:

- Identifying opportunities to leverage spot instances across regions to reduce the cost of AI batch jobs.
- The design of a scheduling policy that draws spot capacity from multiple regions while guaranteeing deadlines.
- `SkyVoyager`, an AI batch job execution system that runs jobs across regions with significant cost savings.
- Open-sourcing spot preemption traces and `SkyVoyager` to stimulate future research in this area.

2 Background and Motivation

2.1 Characteristics of AI Batch Jobs

We first highlight several key characteristics of AI batch jobs.

Deadlines and slack time. AI batch jobs are typically internal workloads that do not directly face end users. Unlike on-

line serving, which prioritizes responsiveness and must meet tight service-level objectives (SLOs), AI batch jobs often run for long periods and do not require immediate response. Examples include model training and large-scale data analytics using AI models. Although they are not latency sensitive, such jobs usually have deadlines and cannot be delayed indefinitely. For instance, model training must finish before a scheduled release, and daily data pipelines must complete before the next cycle begins. In this paper, we assume jobs have an *explicit* deadline specified in advance, and the job must finish before it. We also assume that the job's execution time is shorter than the time available before the deadline, which creates *slack time*, representing the amount of delay the job can tolerate before risking a deadline violation.

Interruptible. Due to their long-running nature, batch jobs are generally designed to be interruptible. They must tolerate unexpected failures without losing significant progress. Training jobs typically perform periodic checkpointing, saving model parameters and optimizer states as recovery points [39, 68]. If a failure occurs, the job resumes from the most recent checkpoint. Batch inference and data processing workflows can often be decomposed into independent units whose outputs are stored incrementally, with the processed data index serving as a lightweight checkpoint. This allows failures to be handled by restarting from unprocessed units rather than re-executing the entire dataset.

Cold start delay. As model sizes grow, batch jobs incur a non-negligible cold start delay both at job launch and during every recovery. This delay includes instance provisioning, dependency installation, and downloading and loading model weights onto GPUs. For stateful jobs such as model training, it also includes transferring job state when recovering in a new location. For example, starting supervised fine-tuning for a 14B-parameter model on 8 GPUs can take 5–10 minutes. These cold start delays significantly affect overall progress and must be accounted for when designing scheduling policy.

2.2 Existing Elastic Systems for AI Batch Jobs

Given these characteristics, we now review existing systems for running batch jobs in the cloud and discuss their limitations and missed opportunities for cost reduction (Table 1). We focus on elastic systems that dynamically provision cloud resources, and treat on-prem clusters and reserved instances as orthogonal, since any workload that exceeds such static capacity needs to be served by elastic cloud resources.

2.2.1 On-Demand Only Systems

The most common approach is to launch on-demand instances, run the job to completion, and then terminate the instances. One example is AWS SageMaker [4], a managed execution system for AI batch jobs (we discuss its managed spot support later). This approach guarantees timely execution before the deadline and avoids most failures, which simplifies system design. However, it is very costly. Because on-demand execution ignores slack, jobs often finish well before their deadlines, while still incurring the full premium of on-demand pricing.

	Deadline Guarantee	Spot Instance	Multi-Region
SageMaker [4]	✓	×	×
Spot Only [4, 14, 52]	×	✓	×
SkyServe [35]	×	✓	✓
Uniform Progress [60]	✓	✓	×
`SkyVoyager`	✓	✓	✓

Table 1: Comparison of systems. Spot Only Systems: SageMaker Managed Spot [4], Parcae [14], and Bamboo [52].

2.2.2 Spot Systems

Prior work [4, 14, 35, 52, 63, 67] uses spot to reduce cost. These systems fall into two categories: (1) batch-oriented spot-only systems and (2) online serving multi-region systems.

Spot-only systems fail to meet deadlines. Several existing systems like SageMaker Managed Spot [4], Parcae [14] and Bamboo [52] rely exclusively on spot instances. This works well when spot availability is high, but execution can be delayed indefinitely when spot capacity becomes scarce. Prior work [35] shows that, in the worst case, a region can run out of spot for 72 hours, during which the job makes no progress.

Online systems target a different workload. Systems such as SkyServe [35] run online LLM serving on spot instances across multiple regions and prioritize responsiveness, provisioning on-demand when spot is insufficient to maintain latency SLOs. Because they process requests immediately and cannot exploit slack, they miss cost-saving opportunities and are unsuitable for batch jobs.

2.2.3 Deadline-Aware Systems

Uniform Progress (UP) [60] is a deadline-aware policy that uses on-demand and spot instances interchangeably. It uses spot when available, falls back to on-demand when capacity is scarce or slack is low, and spreads job progress evenly over time to meet the deadline. While simple, UP ignores temporal variation in spot availability and runs in a single region; when that region has no spot capacity, it must rely on on-demand, pushing cost close to that of always using on-demand (§5.1).

UP assumes that spot availability is sufficiently stable within a single region to allow steady progress. Our analysis in §2.3 shows that real-world availability and lifetime are highly non-uniform across both regions and time, making single-region policies inherently inefficient. Therefore, a more effective policy could leverage multiple regions to access additional spot capacity and further reduce cost [6].

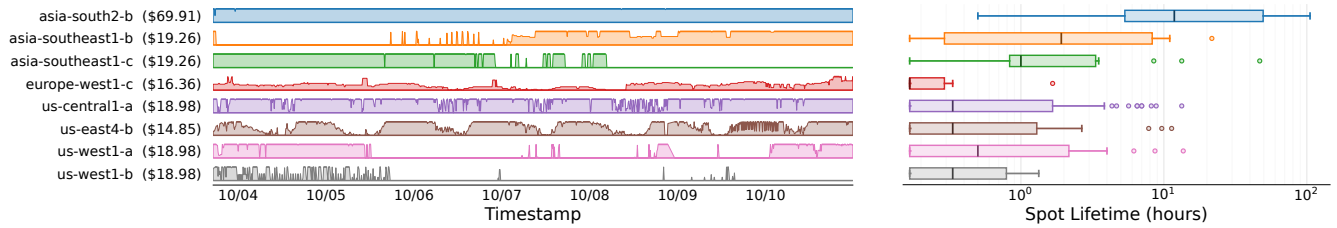


Figure 2: Spot availability and lifetime vary widely across regions. **Left**: availability for 16 `a3-highgpu-1g` (1×H100) instances on GCP over 7 days, shown for 8 representative zones out of the 13 zones we collected. The y-axis shows how many instances are available at each moment. **Right**: box plot of the lifetime distribution, log-scaled. We consider a job that uses 16 instances in a gang-scheduled manner: the entire job is preempted if any single instance is preempted, and it resumes only when the full set of 16 instances becomes available again. We plot the resulting lifetimes under this scenario.

2.3 Heterogeneity of Spot Instances

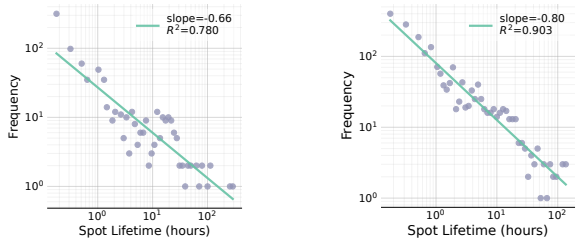
Cloud providers offer a special class of instances, spot instances, as a cost-efficient option. They are typically 3–10× cheaper [31, 35, 60] than regular on-demand instances but come with the risk of being *preempted* at any time by the provider. Spot instances exhibit both spatial heterogeneity across regions and temporal heterogeneity over time, across multiple dimensions. Next, we describe these dimensions and the challenges in harnessing them to reduce cost.

2.3.1 Non-Uniform Spot Availability Patterns

Spot availability represents the periods during which a region has accessible spot capacity. An oracle scheduler with perfect future knowledge of availability could almost always run the job *entirely* on spot instances, although not necessarily within the same region, achieving significant cost reductions. In practice, however, availability patterns vary across regions and fluctuate over time. When availability is unknown ahead of time, it becomes difficult to characterize each region and decide which one is the best region at any moment.

We collect spot availability traces for 16 `a3-highgpu-1g` instances (1×H100) across 13 GCP regions. Figure 2 shows 8 representative examples (complete traces for all 13 zones are in Figure 11). Different regions exhibit distinct availability patterns: some are generally available (`asia-south2-b`), some are mostly available but experience frequent preemptions (`us-central1-a`), and some are largely unavailable (`us-west1-b`). We find that simultaneous preemptions across regions are rare, consistent with observations in [30, 35]. This makes regions complementary: at almost any given time, at least one region has available spot capacity. Prior work [35] shows that the fraction of time in which spot instances are available quickly approaches 99% once more than four regions are included, and we observed similar behavior for 6 regions in our trace.

Beyond cross-region heterogeneity, availability within a single region also exhibits large variance. For example, `asia-southeast1-c` is highly available in the first half of the trace but completely unavailable in the second half. `us-east4-b` shows a clear diurnal pattern, being available during nighttime and unavailable during daytime. These observations indicate that no single perfect region is consistently available (we discuss the limitation of `asia-south2-b` in §2.3.3). As a result, a single-region policy is fundamentally limited, and handling non-uniform spot availability patterns becomes the central challenge.



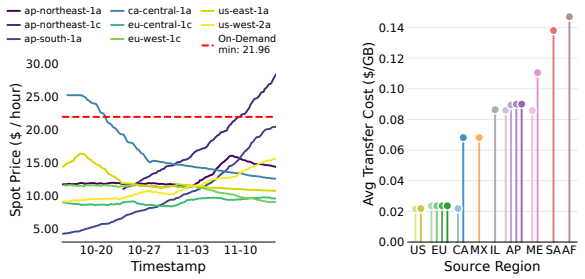
(a) 16×H100 on GCP. (b) 1×V100 on AWS.

Figure 3: Spot lifetime distributions on different accelerators. Scatter points show frequency counts in equal-width $\log_{10}$ bins, and solid lines are least-squares fits in log–log space, showing a clear heavy-tailed pattern [11, 34].

2.3.2 Highly Variable Spot Lifetimes

Beyond availability, spot lifetime is another critical metric. It is defined as the duration for which a spot instance can be used before it is preempted. Because cold start time is non-negligible and does not contribute to job progress, short-lived spot instances provide little value if their lifetime is close to or smaller than the cold start delay. Let d denote the cold start time. If a spot instance has a lifetime t_0, then the effective time during which the job progresses is $t_0 - d$. When t_0 approaches d, the job achieves very low goodput since most of the time is spent restarting rather than computing.

However, lifetime distributions are highly variable, across both regions and time. For example, Figure 2 shows the distribution of spot lifetimes for 8 regions: median lifetimes range from under 1 hour (`europe-west1-c`) to over 20 hours



(a) Spot prices. (b) Egress costs.

Figure 4: Cost differences across regions and time. (a) **Spot prices** for the same instance type. The dashed line shows the cheapest on-demand price. (b) **Egress costs** determined by the source region and ranging from $0.02/GB (US, EU) to $0.14/GB (SA, AF). Abbreviations follow AWS region naming conventions (e.g., AF for Africa, SA for South America).

(`asia-south2-b`), spanning more than an order of magnitude. Selecting a region with a long upcoming spot lifetime is essential because it reduces the number of recoveries and amortizes cold start overhead over longer execution periods.

We also observe that many spot preemptions occur within short time spans, which we refer to as *volatile periods*. These periods produce many short-lived spot instances that become available briefly and are preempted almost immediately. For example, in region `us-central1-a`, 90% of spot preemptions occur within 85 hours of a 15-day period.

Conversely, we observe that spot lifetimes follow a heavy-tailed distribution [27,33]: the longer an instance has survived, the longer its expected remaining lifetime tends to be. As illustrated in Figure 3, the near-linear decay in log–log space ($R^2 \approx 0.78$–0.90, indicating that a power-law model explains 78–90% of the observed variance in lifetime frequency) confirms this pattern: most instances are preempted within a few hours, yet a meaningful tail survives for tens of hours or more.

2.3.3 Pricing Differences and Proactive Migration

Another form of heterogeneity resides in the pricing strategies. As shown in Figure 4a, prices for the same instance type can differ significantly across regions, reaching up to 5×. In most clouds (including AWS [5] and GCP [16]), prices also fluctuate over time and can vary by up to 1.7× within only 12 days. Prior work [35, 60] often overlooks these differences and assumes static pricing, which oversimplifies the problem.

Beyond instance prices, cloud providers impose additional charges for cross-region data transfers. For jobs that maintain large checkpoints, such as training workloads, the checkpoint must be moved whenever the job migrates to a new region. This movement incurs non-negligible *egress* costs [1, 13, 37], which vary depending on the source region and the workload itself. As shown in Figure 4b, egress costs can differ by up to 7× across regions. For example, a 14B-parameter fine-tuning job may need to move about 500GB of replicated checkpoints; this costs roughly $10 from US to Europe but around $40 from Asia to US. Across a range of recent models (32B–1T parameters), we show in §A.2.2 that egress per migration remains a bounded fraction (∼40%) of one hour's compute cost, regardless of model scale.

Migrating to a region with a cheaper spot price may sound appealing, even when the current spot instance has not been preempted. This form of *price-aware proactive migration*, however, is only beneficial when the migration cost is justified by a sufficiently long spot lifetime in the cheaper region. Let the price in region i be p_i, the migration cost between regions i and j be $C_{i,j}$, the expected spot lifetime in region i be t_i, and d be the cold start delay. Suppose the job is currently running on a spot instance in region a. When a cheaper region b becomes available, migrating is beneficial only if $C_{a,b} \leq (p_a - p_b) \cdot (t_b - d)$.

However, a single region rarely offers the best availability, the longest lifetime, and the lowest price simultaneously. For example, in §2.2.3, while mostly available, `asia-south2-b` is 4× more expensive than the cheapest region and approaches the on-demand price. In contrast, `us-east4-b` offers the lowest cost but experiences frequent preemptions and volatile periods. Along with migration cost, trading off among these four metrics creates a vast combinatorial decision space.

3 Design

We now present the design of `SkyVoyager` (Figure 5, Algorithm 1) to fully harness spot instances for AI batch jobs by accounting for spatial and temporal heterogeneity in availability, lifetime, and pricing (§2.3). At a high level, `SkyVoyager` models the job's execution as a sequence of transitions among placement states, each characterized by a region and an instance mode (spot, on-demand, or idle), and continuously evaluates every available option, switching to a more favorable one whenever beneficial. We formulate this per-step greedy strategy as a stochastic optimal control problem (§3.1) and identify its Bellman structure [8], under which the greedy policy is near-optimal when the utility of each option is estimated accurately. The problem therefore reduces to accurate utility estimation (§3.2). The utility of each option decomposes into a progress value, a compute cost, and an amortized migration cost over a predicted spot lifetime. Compute cost, migration cost, and cold-start time are directly observable and require no estimation. For the remaining terms, we employ a principled surrogate to approximate the progress value (§3.3), empirically validating that the resulting policy stays within 6–12% of the omniscient optimal (§5.2). For spot lifetimes, `SkyVoyager` predicts them via survival analysis using a virtual-instance view produced by lightweight probing (§3.4). §3.5 combines these components into the complete scheduling policy.

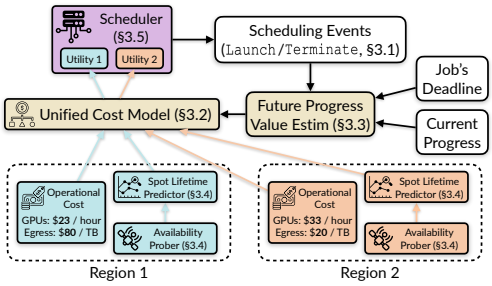


Figure 5: Design overview of `SkyVoyager`.

3.1 Problem Formulation

We consider a single batch job that runs on a fixed group of instances placed in the same region. To maximize compatibility with existing systems, we assume the job is gang-scheduled, so that the entire instance group is treated as an atomic unit and the preemption of any instance is considered a preemption of the whole group. Without loss of generality, we focus on scheduling a single atomic group, as gang scheduling abstracts an instance group as a single atomic resource. This contrasts with autoscaling designs that distribute work across a dynamic number of instances and multiple regions simultaneously (we discuss this further in §7).

The job checkpoints periodically, enabling it to pause and resume in a different region after an interruption. We assume the execution time can be estimated and denote it as P units of work. Whenever the job starts or resumes on a new instance, it incurs a cold-start delay of d time units before it can make progress (§2.1). Let $p(t)$ denote the cumulative job progress by time t, defined as

$$\frac{\mathrm{d}p}{\mathrm{d}t} = \begin{cases} 1 & \text{if the job is ready to run,} \\ 0 & \text{otherwise (e.g., cold start or idle).} \end{cases} \tag{1}$$

The job needs to accumulate enough progress before the deadline, namely $p(T) \geq P$ by time T. Our objective is to minimize the total cost $\mathcal{C}$ under this constraint, which consists of two parts: 1) Compute cost $\mathcal{C}_{\text{compute}} = \int_{m \neq \texttt{idle}} C_{(r,m)}(t)\,\mathrm{d}t$ is the cumulative hourly price over all time the job is running. 2) Migration cost $\mathcal{C}_{\text{migrate}} = \sum_{i=1}^{k} E_{r_{i-1} \to r_i}$ is the sum of egress fees over all k region switches, where $E_{r_i \to r_j} = e_{r_i \to r_j} \cdot S_{\text{ckpt}}$ is the total egress cost per migration, $e_{r_i \to r_j}$ is the per-byte egress rate from region r_i to region r_j, and S_{ckpt} is the checkpoint size ($E_{r \to r} = 0$ since resuming in the same region requires no checkpoint migration).

Let $\mathcal{R}$ denote the set of candidate regions, each offering both spot and on-demand instances. At time t, the system state is $s = (t, p, r, m)$, where p is the current progress, $r \in \mathcal{R}$ is the current checkpoint location, and $m \in \{\texttt{idle}, \texttt{spot}, \texttt{od}\}$ is the instance mode (`od` is on-demand and `idle` means not running on any instance, waiting for a better opportunity). The hourly price for running in mode m in region r at time t is denoted as $C_{(r,m)}(t)$.

State changes occur through three events:

- `Launch`(r,m): attempts to start an instance in region r with mode $m \in \{\texttt{spot}, \texttt{od}\}$. On-demand always succeeds; spot succeeds only if capacity is available, which is unknown until the attempt. A successful launch migrates the checkpoint to r if it is not already there.
- `Terminate`: stops the current instance, setting $m \leftarrow$ `idle` with r unchanged.
- `Preemption`: the cloud reclaims the spot instance, with the same effect as `Terminate`.

Deadline safety rules. Before presenting the main policy, we establish two rules to guarantee deadline completion, adapted from [60] with a multi-region extension.

Thrifty Rule. The job remains idle once $p(t) \geq P$.

Safety Net Rule. When idle with $T - t < P - p(t) + d$, the scheduler falls back to on-demand until completion. This is a feasibility constraint: with only d time for cold start, the job cannot finish unless on-demand begins immediately. In a multi-region setting, the fallback selects the cheapest option:

$$\arg\min_{r \in \mathcal{R}} \left[C_{(r,\texttt{od})}(t) \cdot (P - p(t) + d) + E_{r_0 \to r} \right] \tag{2}$$

where r_0 is the current checkpoint location.

3.2 Unified Cost Model

We first elaborate the Bellman structure of the scheduling problem, then show how the one-step lookahead reduces the optimal action to a per-action utility that depends on a single scalar: the marginal value of progress.

Bellman equation. The scheduling problem defined in §3.1 is a stochastic optimal control problem: the state $s = (t, p, r, m)$ evolves under discrete control actions (`Launch`, `Terminate`) and exogenous stochastic events (`Preemption`). At each scheduling step, the scheduler chooses an action a: continue the current instance, `Launch`(r', m'), or `Terminate` and idle. Let $J^*(s)$ denote the optimal cost-to-go, the minimum expected future cost to complete the job from state s. Following standard dynamic programming [8], $J^*(s)$ satisfies the Bellman optimality equation:

$$J^*(s) \;=\; \min_{a \in \mathcal{A}(s)} \Big\{ \text{cost}(s,a) \;+\; \mathbb{E}\big[J^*(s')\big] \Big\} \tag{3}$$

where $\mathcal{A}(s)$ is the set of feasible actions, s' is the stochastic next state, and $\text{cost}(s,a)$ is the cost of action a in state s. An exact solution is intractable: the action space is combinatorial ($|\mathcal{R}| \times \{\texttt{spot}, \texttt{od}, \texttt{idle}\}$), preemptions are stochastic, and it requires full knowledge of future information.

Per-action utility. Each action $a = (r,m)$ places the job in region r with mode m for an expected duration $\bar{L}_{(r,m)}$ (the predicted spot lifetime from §3.4). Following standard one-step lookahead with renewal amortization, we characterize each action by its average rates over $\bar{L}_{(r,m)}$.

Effective rates. Action a incurs compute cost at rate $C_{(r,m)}(t)$; migrating from r_0 also pays a one-time transfer

cost $E_{r_0 \to r}$. The cost rate combines compute and amortized migration:

$$C_{(r,m)}(t) \;+\; \frac{E_{r_0 \to r}}{\bar{L}_{(r,m)}}. \tag{4}$$

The *effectiveness* (progress rate) $\eta_{(r,m)} \triangleq (\bar{L}_{(r,m)} - d)^+ / \bar{L}_{(r,m)}$ is the fraction of expected lifetime spent doing useful work.

Taylor expansion in rate form. Rather than evaluating J at each candidate's future state, we apply a first-order Taylor expansion, which suffices because each action's progress is a small fraction of the total job P, making higher-order terms negligible for ranking. Over a common evaluation interval $\mathrm{d}t$, action a incurs one-step cost $\mathrm{cost}(s,a) = [\text{eq. (4)}] \cdot \mathrm{d}t$ and advances progress by $\eta_a \, \mathrm{d}t$. Expanding J at the post-action state:

$$J(t{+}\mathrm{d}t,\; p{+}\eta_a \, \mathrm{d}t) \;\approx\; J(s) \;+\; \frac{\partial J}{\partial t}\,\mathrm{d}t \;+\; \frac{\partial J}{\partial p}\,\eta_a \,\mathrm{d}t. \tag{5}$$

Combining. Substituting eqs. (4) and (5) into the one-step lookahead (eq. (3)), the terms $J(s)$ and $\frac{\partial J}{\partial t}\,\mathrm{d}t$ do not depend on a and drop from the ranking; dividing by $\mathrm{d}t$:

$$a^* = \arg\min_a \left\{ C_a + \frac{E_{r_0 \to r}}{\bar{L}_a} + \frac{\partial J}{\partial p}\,\eta_a \right\}.$$

Since $\partial J / \partial p < 0$ (more progress reduces future cost), the minimization reduces to maximizing the *utility* of action (r,m) (full derivation in §A.4):

$$U_{(r,m)} \;\triangleq\; \underbrace{\left(-\frac{\partial J}{\partial p}\right) \cdot \eta_{(r,m)}}_{\substack{\text{Value of} \\ \text{effective progress}}} - \underbrace{C_{(r,m)}(t)}_{\substack{\text{Compute} \\ \text{cost}}} - \underbrace{\frac{E_{r_0 \to r}}{\bar{L}_{(r,m)}}}_{\substack{\text{Amortized} \\ \text{migration cost}}} \tag{6}$$

The scheduler selects $a^* = \arg\max_a U_a$ at each step (§3.5).

The compute cost $C_{(r,m)}$ and migration cost $E_{r_0 \to r}$ are set by the cloud environment; effectiveness η is set by the spot lifetime and cold-start overhead. Short-lived spot instances score poorly because cold start dominates their effectiveness and the fixed migration fee is spread over fewer ticks. For on-demand instances, $\bar{L}_{(r,\mathtt{od})} \to \infty$ (no preemption), so $\eta_{(r,\mathtt{od})} \to 1$ and migration cost is fully amortized: $U_{(r,\mathtt{od})} = (-\partial J/\partial p) - C_{(r,\mathtt{od})}$. For idling, no cost is incurred and no progress is made: $U_{\mathtt{idle}} = 0$. Taking an action is therefore worthwhile only when $U > 0$.

The derivative $-\partial J/\partial p$ measures how much each unit of progress reduces the future cost, separating the value of progress (a global schedule-dependent quantity) from each action's progress output.

3.3 Progress Value Estimation

The only quantity in U (eq. (6)) that is not directly observable is $-\partial J/\partial p$. Computing it exactly requires the intractable J^*; following standard approximate DP [8, 43], we replace J^* with a closed-form surrogate $\hat{J}(t,p)$ and use $-\partial \hat{J}/\partial p$ in its place. When $\hat{J} = J^*$, the resulting greedy policy is provably optimal [8]; the fidelity of $\hat{J}$ therefore directly governs policy quality. We first describe the desirable properties of $\hat{J}$, then present our concrete surrogate and empirically validate that the gap to the true optimal is small.

Desired properties of $\hat{J}(t,p)$.

1. **Completion boundary:** $\hat{J}(t,P) = 0$. Once the job finishes ($p = P$), no future cost remains (Thrifty Rule, §3.1).
2. **Progress barrier:** $\hat{J}(t,p) \to \infty$ when $p \to 0^+$. An infinite surrogate cost at negligible progress ensures that the policy avoids any action leading to such states.
3. **Deadline urgency:** $\hat{J}(t,p) \to \infty$ when $t \to T$ with $p < P$. An infinite cost as the deadline approaches with unfinished work forces the policy to act with increasing urgency. We encode this via the ratio $t/(T-t)$ to ensure that rescaling the timeline does not change the policy.
4. **Cost scaling:** The surrogate must be expressed in the same unit (dollars) as J^*. A natural anchor is the fallback strategy of running on-demand until completion (Safety Net Rule in §3.1): because on-demand is always available, its cost rate $C_{\mathtt{od}} = \min_r C_{(r,\mathtt{od})}$ upper-bounds the per-unit-time cost of any optimal policy. Scaling $\hat{J}$ by $C_{\mathtt{od}}$ grounds the surrogate to this deterministic baseline.

Properties 1–2 characterize a *barrier function* on $p \in (0,P]$. A standard choice is the logarithmic barrier [9,42]: $P\log(P/p) + p - P$, which equals zero at $p = P$ and diverges as $p \to 0^+$. Multiplying by a finite-horizon urgency factor $t/(T-t)$ (property 3) and scaling by $C_{\mathtt{od}}$ (property 4) yields the surrogate we take:

$$\hat{J}(t,p) = C_{\mathtt{od}} \cdot \frac{t}{T-t} \cdot \big[P\log(P/p) + p - P\big]. \tag{7}$$

Shadow price of progress. Differentiating eq. (7) in coordinate p yields the *shadow price of progress*, the marginal value of one unit of progress in dollar terms and the standard costate variable in optimal control [8]:

$$V(t,p) \;=\; -\frac{\partial \hat{J}}{\partial p} \;=\; C_{\mathtt{od}} \cdot \frac{t}{T-t} \cdot \frac{P-p}{p} \;=\; C_{\mathtt{od}} \cdot \frac{\theta(t)}{\tilde{\theta}(t)}, \tag{8}$$

where $\theta(t) = (P-p)/(T-t)$ is the *deadline pressure* (remaining work per remaining time) and $\tilde{\theta}(t) = p/t$ is the *average progress rate*. Their ratio $\theta/\tilde{\theta}$ is a dimensionless *schedule ratio*. When on schedule ($\theta = \tilde{\theta}$), $V = C_{\mathtt{od}}$: the policy is indifferent about on-demand ($U_{\mathtt{od}} = V - C_{\mathtt{od}} = 0$) and accepts any cheaper spot but will not pay the premium cost for on-demand. When the job falls behind ($\theta > \tilde{\theta}$), V rises above $C_{\mathtt{od}}$, making even on-demand worthwhile; when ahead, V drops and the policy becomes selective. Because V depends only on this dimensionless ratio, rescaling the job or the timeline does not change the policy.

DP oracle and empirical validation. To assess how well our surrogate V approximates the true cost-to-go J^*, we compare

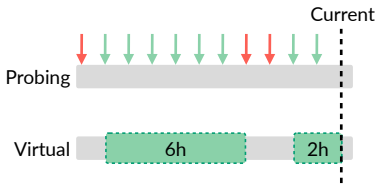


Figure 6: Virtual instance reconstruction from probes. Green arrows: successful probes; red arrows: failed probes. Consecutive successes form virtual lifetime samples (bottom lane).

against the oracle policy with perfect knowledge of future spot availability. On a realized trace ω where all future spot availability is known, the stochasticity in eq. (3) vanishes and the Bellman equation can be solved exactly via backward induction. We call this the *Optimal* policy; it serves as a cost lower bound throughout the evaluation (§5.2.1). Our log-barrier V achieves the lowest aggregate cost gap and the most consistent performance across all evaluated cells (§A.7). Lemma 1 additionally provides a worst-case sensitivity bound from standard approximate DP [49].

3.4 Probing and Lifetime Prediction

The utility (eq. (6)) requires predicted lifetimes $\bar{L}_{(r,m)}$ for each candidate region, which `SkyVoyager` estimates from historical availability observations. Longer-lived spot instances are more valuable because they amortize cold start overhead and migration cost over more productive time. However, collecting lifetime data from every region by running actual instances is prohibitively expensive. `SkyVoyager` maintains a *virtual instance* for each candidate region, realized by lightweight *probing*: periodically launching a spot instance in each region and immediately terminating it if the launch succeeds. Each probe costs one minute of spot compute and reveals whether the region currently has capacity.[1]

Virtual instance. From probe results, launch outcomes, preemption events, and proactive terminations, `SkyVoyager` reconstructs a *virtual instance* trace for each region (Figure 6), as if an instance were continuously running. Allocation failures are strongly correlated with preemption events [60], so a failed probe is treated as a virtual preemption and a successful one as a virtual provisioning. This provides both instantaneous availability signals and the historical lifetime samples needed for prediction, without maintaining a real instance in every region.

Lifetime prediction via survival analysis. From each region's virtual instance trace, `SkyVoyager` extracts historical lifetimes (continuous periods of availability) and applies nonparametric survival analysis, conditioning on the current *age*, the time since the last virtual provisioning, to exploit the heavy-tailed distribution (§2.3.2). Using the Nelson-Aalen estimator [2], `SkyVoyager` estimates the survival function $S(l) = \Pr(L > l)$ and computes the expected remaining lifetime for an instance with current age $a(t)$:

$$\bar{L}(a(t)) = \frac{1}{S(a(t))} \sum_{l_i > a(t)} S(l_i) \tag{9}$$

This estimator naturally captures the heavy-tailed pattern: long-lived instances exhibit lower preemption risk, so the predicted remaining lifetime increases with age. To account for volatile periods where preemptions cluster (§2.3.2), `SkyVoyager` scales the cumulative hazard by a volatility ratio that measures excess preemptions relative to the baseline rate, penalizing regions currently in a volatile period. Full details of the survival analysis and volatility adjustment are in §A.3. `SkyVoyager` sends probes every two hours; §A.2.3 shows that decision quality is robust to this interval.

Algorithm 1 `SkyVoyager` Scheduling Policy

1: **Input:** Job requiring total progress P, deadline T, and candidate regions $\mathcal{R}$
2: $(r_0, m_0) \leftarrow$ (initial region, `idle`)
3: **while** $p(t) < P$ **do** ▷ Every scheduling step
4: **if** SafetyNet(t) **then** ▷ §3.1
5: **Launch** `od` if $m \neq$ `od`; **continue**
6: ProbeRegions($\mathcal{R}$) periodically ▷ §3.4
7: $V \leftarrow$ ProgressValue$(t, T, p(t), P)$ ▷ §3.3
8: **for** each state $s \in \mathcal{R} \times \{$`spot`, `od`, `idle`$\}$ **do**
9: $\bar{L}_s \leftarrow$ PredictLifetime(s) ▷ §3.4
10: $U_s \leftarrow$ CalcUtility$(s, V, \bar{L}_s)$ ▷ §3.2
11: Sort candidate states by U_s descending
12: **for** each $s = (r, m)$ with $U_s > U_{(r_0,m_0)}$ **do**
13: ▷ Migrate if better option found
14: succeeds $\leftarrow$ **Launch** s **or** $m =$ `idle`
15: **if** succeeds **then**
16: **Terminate** (r_0, m_0) if $m_0 \neq$ `idle`
17: $(r_0, m_0) \leftarrow (r, m)$; **break**
18: **Terminate** (r_0, m_0) ▷ Terminate on job completion

3.5 Putting it All Together

Algorithm 1 summarizes the scheduling policy. At each scheduling step, the scheduler first checks whether the safety net is needed (§3.1, lines 4–5). It then periodically probes all regions (§3.4, line 6), computes the progress value V from the current deadline pressure (§3.3, line 7), predicts spot lifetimes using the virtual instance view (§3.4, line 9), and scores each candidate via the unified cost model (§3.2, line 10). Candidates with utility exceeding the current state are attempted in descending order; the scheduler migrates to the first that

[1] We use lightweight probing so that `SkyVoyager` does not rely on any external availability signals. Third-party or provider signals can be used as a drop-in replacement if available.

succeeds.[2] Idling always succeeds; spot launches may fail due to unavailability. This ensures the system continually moves toward the best available option as conditions change.

4 SkyVoyager Implementation

We implemented SkyVoyager, a general deadline-aware batch job execution system that combines spot and on-demand across multiple regions to guarantee deadline completion while minimizing cost. SkyVoyager is workload-agnostic, supporting any workload that periodically checkpoints to a persistent store, and it seamlessly manages checkpoint migration when moving across regions. SkyVoyager is built on top of SkyPilot [65], an open-source multi-cloud system, and adds a scheduler on top of it with ~6,000 lines of code. The implementation supports common training frameworks including PyTorch [68] and Hugging Face Transformers [58].

Scheduler. The scheduler manages the entire lifecycle of the job. It first performs periodic probes to collect the data used in (§3.4). The probe interval is configurable and is set to 2 hours by default, which we find sufficient to capture key characteristics of the spot market while keeping probing overhead reasonable (sections A.2.3 and 5.1). The scheduler then forwards the probe results to the policy and provisions instances according to the policy's decision.

Checkpoint migration. SkyVoyager assumes that the workload periodically checkpoints to a persistent store (e.g., a cloud bucket) and it handles checkpoint migration when a job is moved to another region. Because data must be loaded onto an instance after it is provisioned, SkyVoyager implements a two-stage pipeline to accelerate migration. While the target instance is provisioning, SkyVoyager first copies the checkpoint to an object store in the target region. Once the instance becomes available, SkyVoyager downloads the checkpoint during runtime setup so that the job can resume execution promptly. Cross-region transfers can be further accelerated by cloud-aware overlay networks [24].

5 Evaluation

We evaluate SkyVoyager comprehensively both on real cloud deployments that experience actual preemptions and in simulation using replayed spot traces. We seek to answer the following questions:

- Can SkyVoyager reduce cost in real cloud environments while still completing jobs within their deadlines? (§5.1)
- Can SkyVoyager generalize across different accelerator types and their preemption patterns? (§5.1 and §5.2.2)
- Can SkyVoyager achieve consistently low cost under varying deadline tightness, number of regions, and checkpoint sizes? (§5.2.3 to §A.2.1)
- Can SkyVoyager generalize across geographic deployments and data sovereignty constraints? (§A.2.4)

5.1 End-to-End Experiments

We run end-to-end experiments on AWS to fine-tune LLMs using three different accelerator configurations: 4×L4, 8×A100, and 4×A10G. We launch all baseline systems and SkyVoyager *simultaneously* so they experience the same real-time spot preemption events. The total cost of these experiments is approximately $8,000.

Baselines. We compare SkyVoyager with:

- **Uniform Progress (UP) [60]**: A single-region deadline-aware policy that distributes progress evenly over time and uses on-demand and spot instances interchangeably to guarantee deadline completion.
- **AWS SageMaker Managed Spot (ASM) [4]**: A production-grade batch job execution system that uses spot instances to reduce cost. ASM relies exclusively on spot instances and can draw spot capacity from multiple availability zones within a single region, but only switches zones upon preemption and does not support multi-region.
- **UP(S)**: A multi-region extension of UP that we implemented as a baseline, representing SkyPilot's production failover policy [50]. Upon preemption, it *Switches* the job to a *different* region to avoid volatile periods [35], trying candidate regions sequentially from cheapest to most expensive until one succeeds. Between preemptions, it follows UP's progress distribution strategy to guarantee deadline completion and exploit rule [60], staying in the current region as long as it remains available.[3]

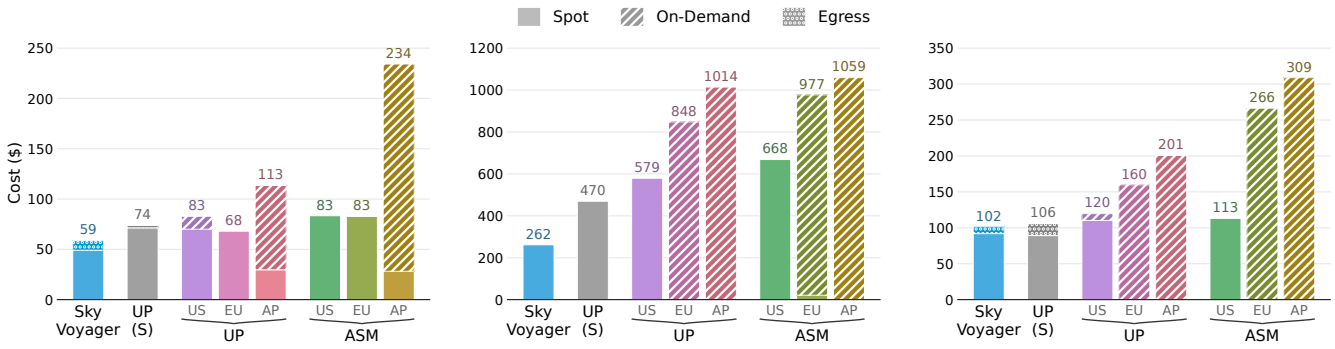
To the best of our knowledge, UP is the only prior system that explicitly exploits slack while providing deadline guarantees for spot jobs. We run UP and ASM separately in each of three regions (us-west-2, eu-central-1, and ap-northeast-1) to study how regional spot availability affects their performance. Because ASM is not deadline-aware, we manually trigger the safety net (§3.1) when needed to meet the deadline.

Workloads. We use Hugging Face Transformers [58] to fine-tune Qwen3 models [62] on the Orca-Math dataset [38]: Qwen3-4B on 4×L4 (g6.12xlarge) and 4×A10G (g5.12xlarge), and Qwen3-14B on 8×A100 (p4d.24xlarge). Each job runs on a single instance and requires about 30 hours of compute with a 45-hour deadline. Checkpoint sizes are 100 GB and 500 GB, respectively. Cold start time, including instance provisioning, environment setup, and checkpoint loading, is approximately 6 minutes.

Results. Figure 7 shows the total cost for each system across the three accelerator configurations. SkyVoyager achieves the lowest cost in all cases: 10–55% savings versus the best single-

[2] In practice, a hysteresis threshold Δ prevents thrashing: the scheduler switches only when $U_{(r,m)} > U_{(r_0,m_0)} + \Delta$.

[3] We also design additional heuristics (UP(A), UP(AP)) to ablate individual components of SkyVoyager in §5.2.



(a) 4×L4 (`g6.12xlarge`) (b) 8×A100 (`p4d.24xlarge`) (c) 4×A10G (`g5.12xlarge`)

Figure 7: End-to-end cost comparison across three accelerator types. `SkyVoyager` achieves the lowest cost in all configurations. Single-region baselines (UP and ASM) show high variance depending on regional spot availability. Only multi-region approaches incur egress costs (dotted region at bar tops). A100 shows minimal egress because the large checkpoint size (500 GB) makes cross-region migration expensive, so the system prefers intra-region zone switches. All jobs complete within their deadlines.

region baseline, 47–69% versus the average single-region baseline, and 4–44% versus the multi-region baseline UP(S). These savings already account for `SkyVoyager` 's overheads: egress costs for cross-region migrations account for up to 16% of total cost (dotted portion in Figure 7), while probing costs are negligible at $1–3 per job. Despite the egress overhead, cross-region migration enables `SkyVoyager` to exploit cheaper spot capacity that would otherwise be inaccessible.

Single region solutions are inherently limited. Single-region baselines face two limitations. First, the ever-changing nature of the spot market means no single region consistently offers the best availability or price. A region that performs well at the start of a job may become unavailable mid-execution, and static region selection cannot adapt to such changes. In our experiments across three representative regions, UP's total cost varies by up to 1.7× on L4 (Figure 7a) and 1.8× on A100 (Figure 7b). While the best region `eu-central-1` for L4 (Figure 7a) achieves low cost with abundant spot capacity, the worst-performing region (`ap-northeast-1`) has no spot available for more than 70% of the time and must fall back to expensive on-demand instances, driving 74% of its total cost. ASM shows even higher variance (up to 2.8× on A10G) because its zone-level failover may land on a more expensive zone (Figure 7c). Second, spot availability patterns differ across accelerator types: `eu-central-1` performs well for L4 but poorly for A100, while `us-west-2` shows the opposite pattern. As a result, region choices informed by one GPU type do not necessarily transfer to another. Given these limitations, even the best-performing single-region baseline (UP in `eu-central-1`) costs 10% more than `SkyVoyager`.

Naive multi-region is not enough. Multi-region approaches achieve lower and more stable costs by drawing spot capacity from multiple regions; both `SkyVoyager` and UP(S) benefit from the aggregated spot resources across regions and complete the job without using any on-demand instances (Figure 7a). However, UP(S) only migrates when preempted. If it lands in an expensive region that happens to have stable availability, it stays there indefinitely. As a result, despite having access to all three regions, UP(S) underperforms the best single-region baseline on L4 (Figure 7a). `SkyVoyager`, in contrast, continuously monitors all candidate zones through probing and proactively migrates when expected savings outweigh migration costs. This enables `SkyVoyager` to save 44% over UP(S) on A100 (Figure 7b). For example, in Figure 8, continuous probing of the cheaper zone `us-east-2b/c` ($1.8/hr) detects an availability window at hours 23–25 and predicts a lifetime of 3.5 hours. Under the unified cost model, this yields a higher utility than `eu-central-1a`, triggering a price-aware proactive migration (□ marker). Such real-time visibility distinguishes `SkyVoyager` from reactive policies like UP(S).

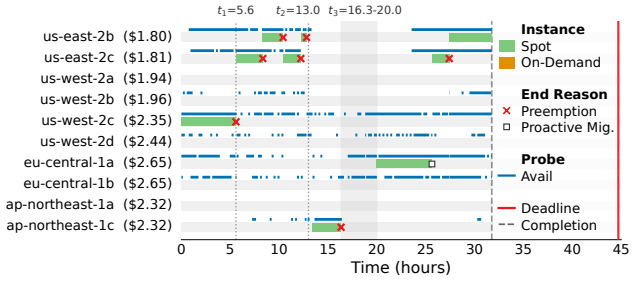


Figure 8: Migration traces for `SkyVoyager` in the L4 experiment. Green bars indicate spot instance usage; blue segments show probe-detected availability; □ markers indicate proactive migration decisions.

`SkyVoyager` sees the whole picture. Unlike heuristics that consider only one metric (e.g., price or availability),

SkyVoyager jointly considers price, predicted lifetime, egress cost, and deadline pressure. Figure 8 highlights three representative behaviors in the L4 run: (1) at t_1=5.6 hr, after preemption in us-west-2c, SkyVoyager proactively migrates to the cheaper us-east-2c ($1.81/hr) because the predicted 4-hour lifetime is long enough to amortize migration cost; (2) at t_2=13.0 hr, after repeated preemptions in us-east-2 drive deadline pressure up ($\theta(t)$=0.73 vs. nominal 0.67), SkyVoyager accepts the more expensive ap-northeast-1c ($2.32/hr) for stable progress; and (3) during t_3=16.3–20.0 hr, all available regions yield non-positive utility, so SkyVoyager waits for better opportunities, and resumes in eu-central-1a once rising deadline pressure makes its price worthwhile. These decisions require combining all four factors in the unified cost model; detailed step-by-step walkthroughs are provided in §A.6.

Summary. E2E experiments confirm that SkyVoyager meets all deadlines while achieving 55% cost savings on average over single-region baselines (up to 70%) through its unified cost model, which balances price, availability, migration cost, and deadline pressure.

5.2 Simulation with Spot Availability Trace

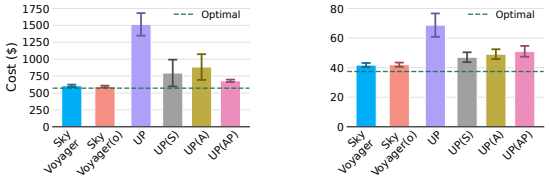
To understand whether SkyVoyager can perform consistently well under diverse conditions, we replay real spot availability traces in simulation. This allows us to systematically study how parameters such as different accelerator types, deadline tightness and number of regions affect the policy at a large scale. Additional sensitivity analyses on checkpoint sizes and data sovereignty requirements are available in §A.

5.2.1 Experimental Setup

Traces. We collect spot availability traces for 16 1×H100 instances (a3-highgpu-1g) across 13 GCP zones over a 14-day period by probing each zone every 10 minutes, at a total cost of approximately $9,000. We also use publicly available traces for 1×V100 instances on AWS [60] for generalizability.

Baselines. We compare SkyVoyager with:

- **Optimal**: The DP oracle (§3.3), an omniscient policy that solves the Bellman equation exactly with full knowledge of future spot availability, serving as a cost lower bound.
- **SkyVoyager (o)**: A variant of SkyVoyager with an oracle for the next spot lifetime in each region.
- **UP**: Single-region Uniform Progress [60], same as in §5.1. We report the average cost across all regions.
- **UP(S)**: Same as in §5.1.
- **UP(A)**: A multi-region extension of UP that uses the same probing method as SkyVoyager (§3.4) and selects regions based on observed availability, defined as the fraction of successful probes in a sliding window of the last 5 samples.



(a) H100 (b) V100

Figure 9: Cost on two independent spot traces: (a) H100 on GCP and (b) V100 on AWS. The green horizontal line indicates the Optimal policy cost. SkyVoyager generalizes across cloud providers and GPU types. We simulate 20 jobs with different start times; error bars show standard error.

- **UP(AP)**: A multi-region extension of UP that selects regions based on availability divided by spot price, balancing availability against cost without lifetime prediction.

Default parameters. Unless otherwise specified, we use a job duration of 100 hours with a 150-hour deadline, a checkpoint size of 50 GB, and a cold start overhead of 6 minutes. We use 8 regions by default.

5.2.2 Generalizability across Accelerators

Figure 9 compares policy performance on two independent spot traces: our H100 trace from GCP and a publicly available V100 trace from AWS [60].

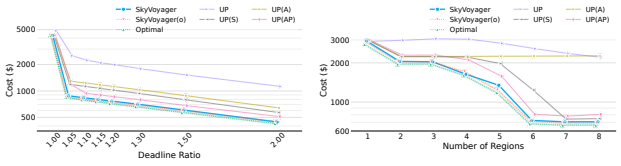
SkyVoyager achieves near-optimal cost on both traces: $610 on H100 (within 11% of Optimal) and $42 on V100 (within 12% of Optimal). This demonstrates that our approach generalizes across cloud providers and GPU types.

We define *selection accuracy* as the fraction of time a policy uses the cheapest available region, capturing its ability to identify and leverage the best option throughout execution. SkyVoyager achieves 83–100% selection accuracy across both traces. In contrast, UP(A) never selects the cheapest region, yielding 0% selection accuracy on both V100 and H100: it always chooses the most available region regardless of price. This explains the cost gap: the unified cost model's price term (§3.2) steers selection toward cost-effective regions, while availability-only heuristics ignore price entirely.

The gap between SkyVoyager and SkyVoyager (o) is under 5% on both traces, with 95–99% region selection overlap, indicating that SkyVoyager's lifetime prediction leads to nearly identical decisions as the oracle. Since SkyVoyager (o) uses an oracle for spot lifetime, this small gap confirms that our survival-analysis lifetime prediction (§3.4) estimates lifetimes accurately enough to make near-optimal decisions.

5.2.3 Impact of Deadline Tightness

Figure 10a shows how cost varies with deadline tightness. The deadline ratio is defined as T/P, where T is the deadline and P is the job duration. Larger ratios provide more flexibility to wait for spot capacity.



(a) Deadline ratio. (b) Number of regions.

Figure 10: Cost vs. (a) deadline ratio, with both axes log-scaled, and (b) number of regions, with the y-axis log-scaled.

At a deadline ratio of 1.0,[4] all policies immediately trigger the safety net (see the deadline safety rules in §3.1) and run entirely on on-demand instances, converging to $4,000–$5,000. As slack increases, `SkyVoyager` tracks Optimal closely, reducing cost to $420 at 2.0× and staying within 10% of Optimal across all tested ratios.

Single-region UP scales poorly, remaining at $1,100 (2.6× `SkyVoyager`) even at 2.0× slack. Multi-region policies scale better by drawing capacity from multiple regions, but still lag `SkyVoyager` by 15–30%. The lag is not due to on-demand fallback, as multi-region policies rarely trigger the safety net. Instead, they select expensive regions: UP(A) greedily chooses high-availability regions regardless of price (up to 3.6× cost in Asia), while UP(AP) and UP(S) react to preemptions without considering predicted lifetime.

5.2.4 Impact of Number of Regions

Figure 10b shows how cost changes as we vary the number of candidate regions from 1 to 8. We incrementally add regions in order of their average spot availability.

With one region, all policies perform similarly ($2,800–$3,000) since there is no region selection to optimize. As regions increase, `SkyVoyager` approaches Optimal: at 8 regions, `SkyVoyager` achieves $707, within 6% of Optimal ($666) and 2% of `SkyVoyager` (o) ($691).

Naive heuristics plateau or underperform. `SkyVoyager` achieves 95% region selection overlap with Optimal, while UP(A) achieves only 0.2%. UP(A) settles in an available but expensive region and never migrates away: in the 8-region scenario, UP(A) selects `asia-south2-b` (95% availability, but 4× the cheapest price) 94% of the time, resulting in 4.8× higher cost than Optimal. UP(S) considers only price and ignores availability, generally underperforming UP(AP). UP(AP) tracks `SkyVoyager` more closely ($806 at 8 regions) by balancing availability and price, but cannot match our unified cost model.

Beyond 6 regions, returns diminish: `SkyVoyager` and Optimal improve steadily up to 6 regions, after which aggregated availability saturates and new regions contribute overlapping availability windows rather than complementary capacity. For users with geographic constraints (§A.2.4), even 2–3 well-chosen regions suffice for significant savings.

[4] In practice, cold start overhead requires a slightly higher ratio.

6 Related Work

Spot instances for training and batch jobs. Spot instances have been used to reduce costs for HPC [54,66], analytics [47, 61], ML training [7,14,25,32,52], caching [56], IaaS [48], and stream processing [64]. These systems maximize throughput but provide no deadline guarantees; relying exclusively on spot means progress halts when capacity disappears [35], and all operate within a single region.

Deadline-aware spot scheduling. UP [60] falls back to on-demand as deadlines approach; Hourglass [26] switches instance configurations based on slack; Proteus [20] introduces tiered reliability for ML training; Snape [63] uses RL for spot/on-demand allocation targeting service SLOs. All are single-region and cannot exploit multi-region spot.

Spot instances for serving. MArk [67], Cocktail [18], Tributary [22], SpotServe [36], and SkyServe [35] use spot for inference with latency SLOs. Unlike batch jobs, serving systems cannot exploit slack and must immediately fall back to on-demand when spot disappears.

7 Discussion

Multi-Cloud Extension. `SkyVoyager` currently operates within a single cloud provider. Extending to multi-cloud could further expand the spot capacity pool, but introduces challenges including heterogeneous APIs, higher cross-cloud migration costs, and provider-specific probing semantics. We leave this as future work.

Autoscaling Integration. `SkyVoyager` uses a fixed number of instance with gang-scheduled preemption for compatibility. Combining multi-region scheduling with elastic autoscaling [7, 25, 36, 44, 52] could yield additional savings, but requires workload-specific support for dynamic parallelism.

Progress Loss After Preemptions. Computation after the last checkpoint is lost upon preemption. `SkyVoyager`'s lifetime prediction could inform adaptive checkpointing [19, 39], where it checkpoints more frequently when the remaining lifetime is predicted to be short. We leave this as future work.

8 Conclusion

`SkyVoyager` shows that exploiting spatial and temporal spot heterogeneity can greatly reduce AI batch job cost while still meeting deadlines. With real-time probing, lifetime prediction, and a unified cost model that captures deadline pressure, and operational cost, `SkyVoyager` continually selects the most cost-effective region and migrates when beneficial. In real-world deployments, `SkyVoyager` reduces costs by 1.25–3.96× compared to existing research and production systems.

References

[1] Network service tiers pricing, 2025. Accessed: 2026-01-10.

[2] Odd Aalen. Nonparametric inference for a family of counting processes. *The Annals of Statistics*, 6(4):701–726, 1978.

[3] Orna Agmon Ben-Yehuda, Muli Ben-Yehuda, Assaf Schuster, and Dan Tsafrir. Deconstructing amazon ec2 spot instance pricing. *ACM Transactions on Economics and Computation (TEAC)*, 1(3):1–20, 2013.

[4] Amazon. Amazon sagemaker. Available at https://aws.amazon.com/sagemaker/, 2025. Accessed: 2026-01.

[5] Amazon Web Services. Amazon EC2 Spot Instances Pricing. https://aws.amazon.com/ec2/spot/pricing/, 2025. Accessed: 2026-01.

[6] Amazon Web Services. Best Practices for EC2 Spot Instances. https://docs.aws.amazon.com/AWSEC2/latest/UserGuide/spot-best-practices.html, 2025. Accessed: 2026-01.

[7] Sanjith Athlur, Nitika Saran, Muthian Sivathanu, Ramachandran Ramjee, and Nipun Kwatra. Varuna: Scalable, low-cost training of massive deep learning models. In *Proceedings of the Seventeenth European Conference on Computer Systems (EuroSys '22)*, pages 472–487. ACM, 2022.

[8] Dimitri P. Bertsekas. *Dynamic Programming and Optimal Control*, volume 1. Athena Scientific, Belmont, MA, 4th edition, 2012.

[9] Stephen Boyd and Lieven Vandenberghe. *Convex Optimization*. Cambridge University Press, 2004.

[10] Tom B. Brown, Benjamin Mann, Nick Ryder, Melanie Subbiah, Jared Kaplan, Prafulla Dhariwal, Arvind Neelakantan, Pranav Shyam, Girish Sastry, Amanda Askell, Sandhini Agarwal, Ariel Herbert-Voss, Gretchen Krueger, Tom Henighan, Rewon Child, Aditya Ramesh, Daniel M. Ziegler, Jeffrey Wu, Clemens Winter, Christopher Hesse, Mark Chen, Eric Sigler, Mateusz Litwin, Scott Gray, Benjamin Chess, Jack Clark, Christopher Berner, Sam McCandlish, Alec Radford, Ilya Sutskever, and Dario Amodei. Language models are few-shot learners, 2020.

[11] Tanujit Chakraborty, Swarup Chattopadhyay, Suchismita Das, Uttam Kumar, and J. Senthilnath. Searching for heavy-tailed probability distributions for modeling real-world complex networks. *IEEE Access*, 10:115092–115107, 2022.

[12] Aakanksha Chowdhery, Sharan Narang, Jacob Devlin, Maarten Bosma, Gaurav Mishra, Adam Roberts, Paul Barham, Hyung Won Chung, Charles Sutton, Sebastian Gehrmann, et al. Palm: Scaling language modeling with pathways. *Journal of Machine Learning Research*, 24(240):1–113, 2023.

[13] CloudZero. Aws egress costs explained: How to reduce spend, 7 2024. Accessed: 2026-01-10.

[14] Jiangfei Duan, Ziang Song, Xupeng Miao, Xiaoli Xi, Dahua Lin, Harry Xu, Minjia Zhang, and Zhihao Jia. Parcae: proactive, liveput-optimized dnn training on preemptible instances. In *Proceedings of the 21st USENIX Symposium on Networked Systems Design and Implementation*, NSDI'24, USA, 2024. USENIX Association.

[15] Nnamdi Ekwe-Ekwe and Adam Barker. Location, location, location: Exploring Amazon EC2 spot instance pricing across geographical regions. In *Proceedings of the 18th IEEE/ACM International Symposium on Cluster, Cloud and Grid Computing (CCGrid '18)*, pages 370–373. IEEE, 2018.

[16] Google Cloud. Spot VMs Pricing. https://cloud.google.com/spot-vms/pricing, 2025. Accessed: 2026-01.

[17] Aaron Grattafiori et al. The llama 3 herd of models, 2024.

[18] Jashwant Raj Gunasekaran, Prashanth Thinakaran, Deepak Narayanan, Ramanathan Kannan, Gabriel Parmer, Prakash Mysore, and Anand Sivasubramaniam. Cocktail: A multidimensional optimization for model serving in cloud. In *19th USENIX Symposium on Networked Systems Design and Implementation (NSDI 22)*, pages 1041–1057. USENIX Association, 2022.

[19] Tanmaey Gupta, Sanjeev Krishnan, Rituraj Kumar, Abhishek Vijeev, Bhargav Gulavani, Nipun Kwatra, Ramachandran Ramjee, and Muthian Sivathanu. Just-in-time checkpointing: Low cost error recovery from deep learning training failures. In *Proceedings of the Nineteenth European Conference on Computer Systems*, pages 1110–1125, 2024.

[20] Aaron Harlap, Alexey Tumanov, Andrew Chung, Gregory R. Ganger, and Phillip B. Gibbons. Proteus: Agile ML elasticity through tiered reliability in dynamic resource markets. In *Proceedings of the Twelfth European Conference on Computer Systems (EuroSys '17)*, pages 589–604. ACM, 2017.

[21] Jordan Hoffmann, Sebastian Borgeaud, Arthur Mensch, Elena Buchatskaya, Trevor Cai, Eliza Rutherford, Diego de Las Casas, Lisa Anne Hendricks, Johannes Welbl,

Aidan Clark, Tom Hennigan, Eric Noland, Katie Millican, George van den Driessche, Bogdan Damoc, Aurelia Guy, Simon Osindero, Karen Simonyan, Erich Elsen, Jack W. Rae, Oriol Vinyals, and Laurent Sifre. Training compute-optimal large language models, 2022.

[22] Jiawei Hu, Sayan Ghosh, Wuxinlin Jiang, Wenqi Zheng, Ling Zhuo, Narayanan Natarajan, Abhishek Chandra, and Ramesh Kannan. Tributary: Spot-dancing for elastic cloud services. In *Proceedings of the Eighteenth European Conference on Computer Systems (EuroSys '23)*, pages 410–426. ACM, 2023.

[23] Ahmed Issaoui, Jonas Örtensjö, and M. Sirajul Islam. Exploring the General Data Protection Regulation (GDPR) compliance in cloud services: insights from Swedish public organizations on privacy compliance. *Future Business Journal*, 9(1):107, 2023.

[24] Paras Jain, Sam Kumar, Sarah Wooders, Shishir G. Patil, Joseph E. Gonzalez, and Ion Stoica. Skyplane: Optimizing transfer cost and throughput using cloud-aware overlays. In *20th USENIX Symposium on Networked Systems Design and Implementation (NSDI 23)*, pages 1375–1389. USENIX Association, 2023.

[25] Insu Jang, Zhenning Yang, Zhen Zhang, Xin Jin, and Mosharaf Chowdhury. Oobleck: Resilient distributed training of large models using pipeline templates. In *Proceedings of the 29th ACM Symposium on Operating Systems Principles (SOSP '23)*, pages 382–395. ACM, 2023.

[26] Pedro Joaquim, Manuel Bravo, Luis Rodrigues, and Miguel Matos. Hourglass: Leveraging transient resources for time-constrained graph processing in the cloud. In *Proceedings of the Fourteenth EuroSys Conference (EuroSys '19)*, pages 1–16, 2019.

[27] JCS Kadupitiya, Vikram Jadhao, and Prateek Sharma. Modeling the temporally constrained preemptions of transient cloud vms. In *Proceedings of the 29th International Symposium on High-Performance Parallel and Distributed Computing (HPDC)*, pages 41–52, 2020.

[28] Jared Kaplan, Sam McCandlish, Tom Henighan, Tom B. Brown, Benjamin Chess, Rewon Child, Scott Gray, Alec Radford, Jeffrey Wu, and Dario Amodei. Scaling laws for neural language models, 2020.

[29] Ali Khajeh-Hosseini, David Greenwood, and Ian Sommerville. Cloud migration: A case study of migrating an enterprise it system to iaas. In *2010 IEEE 3rd International Conference on cloud computing*, pages 450–457. IEEE, 2010.

[30] KyungHwan Kim and Kyungyong Lee. Making cloud spot instance interruption events visible. In *Proceedings of the ACM Web Conference 2024 (WWW '24)*, pages 3490–3501. ACM, 2024.

[31] Dinesh Kumar, Gaurav Baranwal, Zahid Raza, and Deo Prakash Vidyarthi. A survey on spot pricing in cloud computing. *Journal of Network and Systems Management*, 26(4):809–856, 2018.

[32] Kyungyong Lee and Myungjun Son. DeepSpotCloud: Leveraging cross-region GPU spot instances for deep learning. In *2017 IEEE 10th International Conference on Cloud Computing (CLOUD)*, pages 98–105. IEEE, 2017.

[33] Sungjae Lee, Jaeil Hwang, and Kyungyong Lee. Spotlake: Diverse spot instance dataset archive service. In *2022 IEEE International Symposium on Workload Characterization (IISWC)*, pages 242–255. IEEE, 2022.

[34] Will E. Leland, Murad S. Taqqu, Walter Willinger, and Daniel V. Wilson. On the self-similar nature of ethernet traffic. In *Conference Proceedings on Communications Architectures, Protocols and Applications*, SIGCOMM '93, page 183–193, New York, NY, USA, 1993. Association for Computing Machinery.

[35] Ziming Mao, Tian Xia, Zhanghao Wu, Wei-Lin Chiang, Tyler Griggs, Romil Bhardwaj, Zongheng Yang, Scott Shenker, and Ion Stoica. Skyserve: Serving ai models across regions and clouds with spot instances. In *Proceedings of the Twentieth European Conference on Computer Systems*, EuroSys '25, pages 159–175, New York, NY, USA, 2025. Association for Computing Machinery.

[36] Xupeng Miao, Chunan Shi, Jiangfei Duan, Xiaoli Xi, Dahua Lin, Bin Cui, and Zhihao Jia. Spotserve: Serving generative large language models on preemptible instances. In *Proceedings of the 29th ACM International Conference on Architectural Support for Programming Languages and Operating Systems, Volume 2*, ASPLOS '24, pages 1112–1127, New York, NY, USA, 2024. Association for Computing Machinery.

[37] Microsoft Azure Networking Team. A guide to azure data transfer pricing, 2 2025. Accessed: 2026-01-10.

[38] Arindam Mitra, Hamed Khanpour, Corby Rosset, and Ahmed Awadallah. Orca-math: Unlocking the potential of slms in grade school math, 2024.

[39] Jayashree Mohan, Amar Phanishayee, and Vijay Chidambaram. CheckFreq: Frequent, Fine-Grained DNN checkpointing. In *19th USENIX Conference on File and Storage Technologies (FAST 21)*, pages 203–216. USENIX Association, 2021.

[40] Deepak Narayanan, Mohammad Shoeybi, Jared Casper, Patrick LeGresley, Mostofa Patwary, Vijay Korthikanti, Dmitri Vainbrand, Prethvi Kashinkunti, Julie Bernauer, Bryan Catanzaro, et al. Efficient large-scale language model training on gpu clusters using megatron-lm. In *Proceedings of the international conference for high performance computing, networking, storage and analysis*, pages 1–15, 2021.

[41] Michael J. Neely. *Stochastic Network Optimization with Application to Communication and Queueing Systems*. Synthesis Lectures on Communication Networks. Morgan & Claypool, 2010.

[42] Yurii Nesterov and Arkadii Nemirovski. *Interior-Point Polynomial Algorithms in Convex Programming*, volume 13 of *Studies in Applied and Numerical Mathematics*. SIAM, 1994.

[43] Warren B. Powell. *Approximate Dynamic Programming: Solving the Curses of Dimensionality*. Wiley Series in Probability and Statistics. Wiley, 2nd edition, 2011.

[44] Aurick Qiao, Sang Keun Choe, Suhas Jayaram Subramanya, Willie Neiswanger, Qirong Ho, Hao Zhang, Gregory R. Ganger, and Eric P. Xing. Pollux: Co-adaptive cluster scheduling for goodput-optimized deep learning. In *15th USENIX Symposium on Operating Systems Design and Implementation (OSDI 21)*, pages 1–18. USENIX Association, 2021.

[45] Samyam Rajbhandari, Jeff Rasley, Olatunji Ruwase, and Yuxiong He. Zero: Memory optimizations toward training trillion parameter models. In *SC20: International Conference for High Performance Computing, Networking, Storage and Analysis*, pages 1–16. IEEE, 2020.

[46] Charles Reiss, Alexey Tumanov, Gregory R Ganger, Randy H Katz, and Michael A Kozuch. Heterogeneity and dynamicity of clouds at scale: Google trace analysis. In *Proceedings of the third ACM symposium on cloud computing*, pages 1–13, 2012.

[47] Prateek Sharma, Tian Guo, Xin He, David Irwin, and Prashant Shenoy. Flint: Batch-interactive data-intensive processing on transient servers. In *Proceedings of the Eleventh European Conference on Computer Systems (EuroSys '16)*. ACM, 2016.

[48] Prateek Sharma, Stephen Lee, Tian Guo, David Irwin, and Prashant Shenoy. SpotCheck: Designing a derivative IaaS cloud on the spot market. In *Proceedings of the Tenth European Conference on Computer Systems (EuroSys '15)*, 2015.

[49] Satinder P. Singh and Richard C. Yee. An upper bound on the loss from approximate optimal-value functions. *Machine Learning*, 16(3):227–233, 1994.

[50] SkyPilot. SkyPilot task YAML specification: Recovery strategies. https://docs.skypilot.co/en/latest/reference/yaml-spec.html, 2025. Accessed: 2026-01.

[51] Byung Chul Tak, Bhuvan Urgaonkar, and Anand Sivasubramaniam. To move or not to move: The economics of cloud computing. In *3rd USENIX Workshop on Hot Topics in Cloud Computing (HotCloud 11)*, 2011.

[52] John Thorpe, Pengzhan Zhao, Jonathan Eyolfson, Yifan Qiao, Zhihao Jia, Minjia Zhang, Ravi Netravali, and Guoqing Harry Xu. Bamboo: Making preemptible instances resilient for affordable training of large DNNs. In *20th USENIX Symposium on Networked Systems Design and Implementation (NSDI 23)*, pages 497–513, Boston, MA, April 2023. USENIX Association.

[53] Hugo Touvron, Louis Martin, Kevin Stone, Peter Albert, Amjad Almahairi, Yasmine Babaei, Nikolay Bashlykov, Soumya Batra, Prajjwal Bhargava, Shruti Bhosale, et al. Llama 2: Open foundation and fine-tuned chat models. *arXiv preprint arXiv:2307.09288*, 2023.

[54] William Voorsluys and Rajkumar Buyya. Reliable provisioning of spot instances for compute-intensive applications. In *Proceedings of the 26th IEEE International Conference on Advanced Information Networking and Applications (AINA)*, pages 542–549. IEEE, 2012.

[55] Borui Wan, Mingji Han, Yiyao Sheng, Yanghua Peng, Haibin Lin, Mofan Zhang, Zhichao Lai, Menghan Yu, Junda Zhang, Zuquan Song, et al. {ByteCheckpoint}: A unified checkpointing system for large foundation model development. In *22nd USENIX Symposium on Networked Systems Design and Implementation (NSDI 25)*, pages 559–578, 2025.

[56] Cheng Wang, Bhuvan Urgaonkar, Aayush Gupta, George Kesidis, and Qianlin Liang. Exploiting spot and burstable instances for improving the cost-efficacy of in-memory caches on the public cloud. In *Proceedings of the Twelfth European Conference on Computer Systems (EuroSys '17)*, pages 620–634, 2017.

[57] Zhuang Wang, Zhen Jia, Shuai Zheng, Zhen Zhang, Xinwei Fu, TS Eugene Ng, and Yida Wang. Gemini: Fast failure recovery in distributed training with in-memory checkpoints. In *Proceedings of the 29th Symposium on Operating Systems Principles*, pages 364–381, 2023.

[58] Thomas Wolf, Lysandre Debut, Victor Sanh, Julien Chaumond, Clement Delangue, Anthony Moi, Pierric Cistac, Tim Rault, Remi Louf, Morgan Funtowicz, Joe Davison, Sam Shleifer, Patrick von Platen, Clara Ma, Yacine Jernite, Julien Plu, Canwen Xu, Teven Le Scao, Sylvain Gugger, Mariama Drame, Quentin Lhoest, and

Alexander Rush. Transformers: State-of-the-art natural language processing. In *Proceedings of the 2020 Conference on Empirical Methods in Natural Language Processing: System Demonstrations*, pages 38–45. Association for Computational Linguistics, 2020.

[59] Yongji Wu, Wenjie Qu, Xueshen Liu, Tianyang Tao, Yifan Qiao, Zhuang Wang, Wei Bai, Yuan Tian, Jiaheng Zhang, Z. Morley Mao, Matthew Lentz, Danyang Zhuo, and Ion Stoica. Lazarus: Resilient and elastic training of mixture-of-experts models, 2025.

[60] Zhanghao Wu, Wei-Lin Chiang, Ziming Mao, Zongheng Yang, Eric Friedman, Scott Shenker, and Ion Stoica. Can't be late: Optimizing spot instance savings under deadlines. In *21st USENIX Symposium on Networked Systems Design and Implementation (NSDI 24)*, pages 185–203, Santa Clara, CA, April 2024. USENIX Association.

[61] Yan Yan, Yongqiang Gao, Yang Chen, Zonghua Guo, Binjie Chen, and Thomas Moscibroda. TR-Spark: Transient computing for big data analytics. In *Proceedings of the Seventh ACM Symposium on Cloud Computing (SoCC '16)*, pages 484–496. ACM, 2016.

[62] An Yang et al. Qwen3 technical report, 2025.

[63] Fangkai Yang, Lu Wang, Zhenyu Xu, Jue Zhang, Liqun Li, Bo Qiao, Camille Couturier, Chetan Bansal, Soumya Ram, Si Qin, Zhen Ma, Inigo Goiri, Eli Cortez, Terry Yang, Victor Ruhle, Saravan Rajmohan, Qingwei Lin, and Dongmei Zhang. Snape: Reliable and low-cost computing with mixture of spot and on-demand vms. In *Proceedings of the 28th ACM International Conference on Architectural Support for Programming Languages and Operating Systems (ASPLOS '23)*, pages 631–643. ACM, 2023.

[64] Youngseok Yang, Geon-Woo Kim, Won Wook Song, Yunseong Lee, Andrew Chung, Zhengping Qian, Brian Cho, and Byung-Gon Chun. Pado: A data processing engine for harnessing transient resources in datacenters. In *Proceedings of the Twelfth European Conference on Computer Systems (EuroSys '17)*, pages 575–588, 2017.

[65] Zongheng Yang, Zhanghao Wu, Michael Luo, Wei-Lin Chiang, Romil Bhardwaj, Woosuk Kwon, Siyuan Zhuang, Frank Sifei Luan, Gautam Mittal, Scott Shenker, and Ion Stoica. SkyPilot: An intercloud broker for sky computing. In *20th USENIX Symposium on Networked Systems Design and Implementation (NSDI 23)*, pages 437–455, Boston, MA, April 2023. USENIX Association.

[66] Sangho Yi, Derrick Kondo, and Artur Andrzejak. Reducing costs of spot instances via checkpointing in the Amazon Elastic Compute Cloud. In *2012 IEEE 3rd International Conference on Cloud Computing (CLOUD)*, pages 236–243. IEEE, 2010.

[67] Chengliang Zhang, Minchen Yu, Wei Wang, and Feng Yan. Mark: Exploiting cloud services for cost-effective, slo-aware machine learning inference serving. In *2019 USENIX Annual Technical Conference (USENIX ATC 19)*, pages 1049–1062. USENIX Association, 2019.

[68] Yanli Zhao, Andrew Gu, Rohan Varma, Liang Luo, Chien-Chin Huang, Min Xu, Less Wright, Hamid Shojanazeri, Myle Ott, Sam Shleifer, Alban Desmaison, Can Balioglu, Pritam Damania, Bernard Nguyen, Geeta Chauhan, Yuchen Hao, and Shen Li. Pytorch fsdp: Experiences on scaling fully sharded data parallel. *Proceedings of the VLDB Endowment*, 16(12):3848–3860, 2023.

A Additional Results

A.1 Full Spot Availability Traces

Figure 11 shows the complete spot availability traces for all 13 GCP zones, extending the 8 representative zones shown in Figure 2. The additional zones further illustrate the diversity of availability patterns across regions.

A.2 Additional Simulation Results

The experiments in the remaining subsections use the same simulation setup, traces, baselines, and default parameters as described in §5.2.1.

A.2.1 Impact of Checkpoint Size

Checkpoint size affects migration cost since egress cost is proportional to the amount of data transferred. Figure 12 varies checkpoint size from 0 to 4 TB to cover workloads ranging from batch inference with small index state to large-scale training (full model and optimizer state).

Single-region UP is unaffected by checkpoint size since it never migrates across regions. `SkyVoyager` scales gracefully: at 4 TB, `SkyVoyager` reaches $850, 57% cheaper than UP at $2,000, by reducing migration frequency 70% compared to UP(AP)—preferring to wait for spot recovery rather than incur expensive cross-region transfers. The slight cost decrease from 200 GB to 500 GB for `SkyVoyager` is within simulation variance.

Heuristics that react to frequently changing signals suffer at large checkpoints. UP(AP) exhibits the steepest cost growth: availability shifts as new probes arrive, and dividing by price amplifies these fluctuations in cheap regions, so a small availability change can flip the ratio ranking and trigger a region switch. UP(A) is more stable because high-availability regions tend to remain high. UP(S) is most stable since prices rarely change, so it returns to the same cheap regions.

A.2.2 Migration Cost Relative to Compute Cost

A key concern for multi-region scheduling is whether cross-region migration cost grows prohibitively with model size. Figure 13 plots migration egress cost against hourly spot compute cost for seven recent open-weight models spanning 32B to 1T total parameters, including both dense (Qwen3-32B, Llama 3.1 405B) and mixture-of-experts architectures (Llama 4 Scout, Qwen3-235B, DeepSeek-V3, GLM-5, Kimi K2.5).

Both quantities scale linearly with total parameter count—compute because fixed-size GPUs are added proportionally to parameters, egress because the checkpoint grows proportionally too—so in the ideal case their ratio is a constant that depends only on the GPU's $/GB-VRAM efficiency, not on model size. For H100 this ratio is 41.8% (one migration = 25 minutes of H100 compute); for H200 it rises to 69.6% and for B200 to 51.3%. Per-GPU differences are visible in the slopes: the H200 line is steepest because its spot price ($18/hr) is nearly identical to H100's ($17/hr) while VRAM is 1.76× larger—fewer GPUs per model make the compute denominator smaller without changing egress. B200 sits between H100 and H200 because its 2.24× VRAM is partially offset by a higher $31/hr node price. In practice `SkyVoyager` will use slightly more compute than the ideal line because real allocations round up to whole GPUs and whole 8-GPU nodes, but the bound from the ideal ratio still holds: egress is capped by the GPU's VRAM-to-price efficiency, not by model size.

A.2.3 Impact of Probe Interval

The probe interval controls how frequently `SkyVoyager` launches background probes to monitor spot availability across regions (§3.4). Figure 14 varies the interval from 10 minutes to 16 hours.

Compute cost (including egress) stays nearly constant across all intervals, varying by only 6% ($1,546–$1,632). This indicates that `SkyVoyager`'s decision quality is robust to probe frequency: even with 16-hour intervals, the scheduler makes comparable migration decisions because the survival analysis model extrapolates well from sparse observations.

Probing overhead, however, grows rapidly at short intervals. Each probe launches a 1-minute spot instance in every candidate region, so probing all 13 regions every 10 minutes costs $2,011 per job—56% of total cost. At the default 2-hour interval, probe cost drops to $170 (10% of total), and the slight compute cost increase from stale information is negligible. Beyond 4 hours, further reductions in probe cost are marginal while compute cost begins to rise as lifetime predictions become less accurate.

A.2.4 Impact of Data Sovereignty Requirements

Data sovereignty requirements (e.g., GDPR [23]) may restrict jobs to specific geographic regions to prevent data from leaving a designated area. We simulate such constraints by limiting candidate regions to the same continent. Figure 15 evaluates `SkyVoyager` under four configurations: US-only (3 regions), Europe-only (2 regions),[5] Asia-only (3 regions), and Global (all 13 regions). `SkyVoyager` performs well even with geographic constraints: $700 in the US (3 regions), close to the Global optimum of $650; $950 in Europe (2 regions), still 52% cheaper than UP; and $870 in Asia (3 regions), 68% cheaper than UP. Even with only 2–3 regions, `SkyVoyager` achieves significant savings, confirming that a modest number of well-chosen regions suffices (§5.2.4). Asia shows the highest variance due to heterogeneous availability patterns. UP(A) performs particularly poorly ($3,038, 3.6× Optimal) by selecting the most expensive region (`asia-south2-b`) 100%

[5] Due to quota limits, we only have access to two European regions.

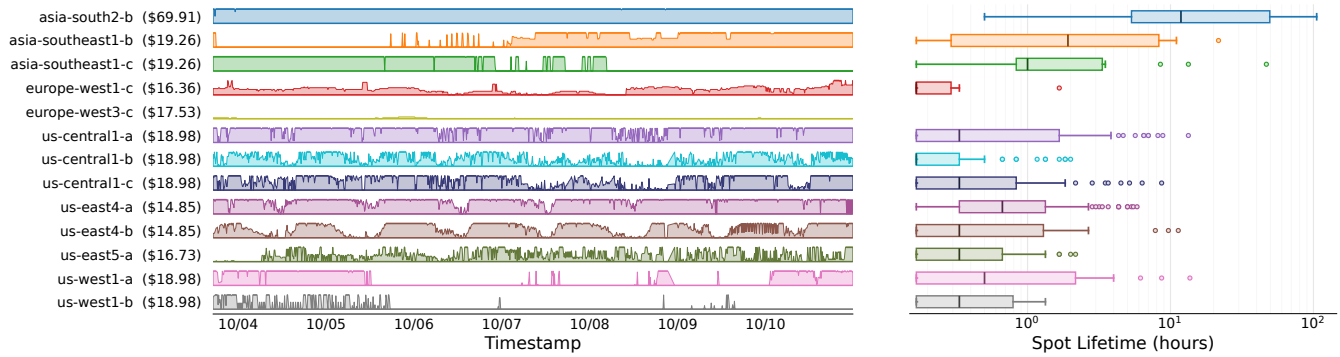


Figure 11: Complete spot availability traces and lifetime distributions for all 13 GCP zones. **Left**: availability for 16 `a3-highgpu-1g` (1×H100) instances over 7 days. **Right**: box plot of the spot lifetime distribution (log-scaled). This is the full version of Figure 2, which shows 8 representative zones.

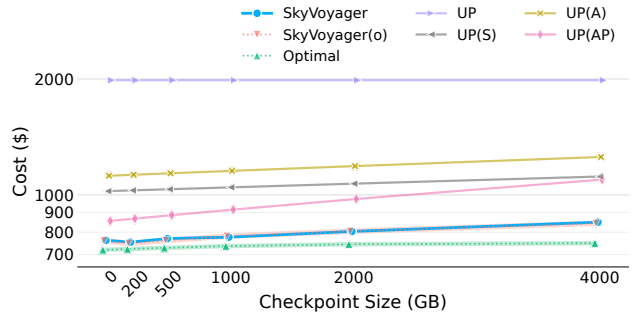


Figure 12: Cost with respect to varying checkpoint sizes. Larger checkpoints increase migration cost, reducing the benefit of cross-region scheduling. `SkyVoyager` scales gracefully by amortizing migration cost over predicted lifetimes, while heuristics like UP(AP) suffer from frequent migrations.

of the time, achieving 0% selection accuracy. In contrast, `SkyVoyager` achieves 100% selection accuracy by consistently selecting the cheapest region (`asia-southeast1-b`), reaching cost within 0.1% of Optimal.

A.3 Survival Analysis Details

This section provides full details of the survival analysis used for spot lifetime prediction (§3.4).

Virtual instance construction. For each region r, `SkyVoyager` maintains a virtual instance from observations (t_i, o_i), where $o_i \in \{0,1\}$ indicates availability. Observations arise from four sources: (1) probes, (2) `Launch` operations, (3) preemption events, and (4) `Terminate` operations when proactively migrating away. A transition from $o_{i-1} = 1$ to $o_i = 0$ is treated as a virtual preemption; the reverse is treated as a virtual provisioning. Each continuous period of $o_i = 1$ constitutes a historical lifetime sample.

Nelson-Aalen estimator. From the virtual instance trace, `SkyVoyager` extracts historical lifetimes. A lifetime begins when a probe or launch succeeds and ends either with an observed preemption or is right-censored when `SkyVoyager` proactively migrates away. Let $e(l)$ denote the number of preemptions at lifetime l and $c(l)$ the number of right-censored observations. The hazard rate is estimated using the Nelson-Aalen estimator [2]:

$$h(l) = \frac{e(l)}{n(l)} \text{ with } n(l) = \sum_{x \geq l} (e(x) + c(x)) \tag{10}$$

where $n(l)$ is the at-risk population. This estimator is non-parametric and naturally captures the heavy-tailed pattern: long-lived instances exhibit lower preemption risk, so $h(l)$ decreases with l.

The survival function is $S(l) = \exp(-\sum_{l_i \leq l} h(l_i))$, giving $\Pr(L > l)$. For an instance with current age $a(t)$, the expected remaining lifetime is computed via eq. (9).

Volatility adjustment. To detect volatile periods (§2.3.2) where preemptions cluster, `SkyVoyager` compares observed preemptions against the baseline hazard rate. For a time window W ending at the current time, the volatility ratio $\gamma_W = e_W / \sum_{t \in W} h(a(t))$ measures how many more preemptions occurred than expected. Taking $\gamma^* = \max_W \gamma_W$ over all window lengths conservatively captures the most severe clustering at any time scale. The survival curve is then adjusted by scaling the cumulative hazard: $\tilde{S}(l) = \exp(-\gamma^* \cdot H(l))$, penalizing regions that are currently in a volatile period.

A.4 Derivation of the Utility Function

We derive eq. (6) from a one-step Bellman expansion of the approximate cost-to-go $\hat{J}(t, p, r)$ defined in §3.2. The derivation has three steps: defining effective rates via renewal amortiza-

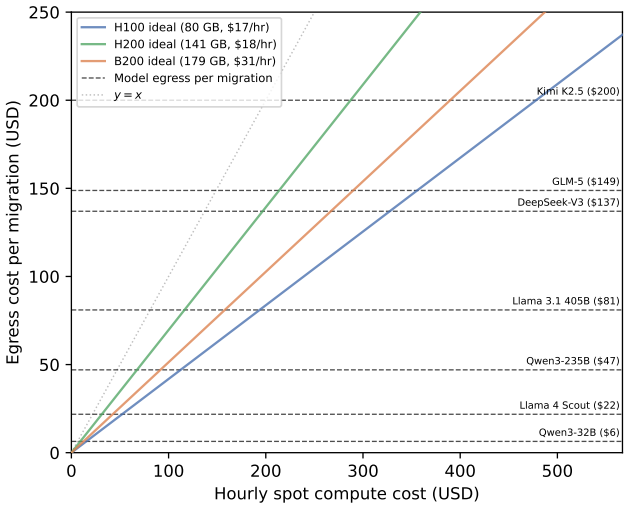


Figure 13: Egress cost per migration vs. hourly spot compute cost for recent open-weight models (32B–1T parameters) on three GPU generations. Both quantities derive from total parameter count: compute from memory-based GPU scaling (18 bytes/param for mixed-precision training with Adam) priced at current AWS spot rates, and egress from the full training checkpoint (10 bytes/param) at $0.02/GB inter-region. Each *horizontal dashed line* marks a model's fixed egress cost per migration (independent of GPU choice). Each *solid diagonal* shows the ideal-packing cost on one GPU generation (slope = egress/compute in USD/USD). Where a horizontal line meets a diagonal is the operating point for that model on that GPU. All lines stay below the $y{=}x$ reference: even on H100 (shallowest slope, 41.8%), one migration costs at most ∼25 minutes of one hour's compute, so egress remains a bounded fraction of hourly compute regardless of model scale.

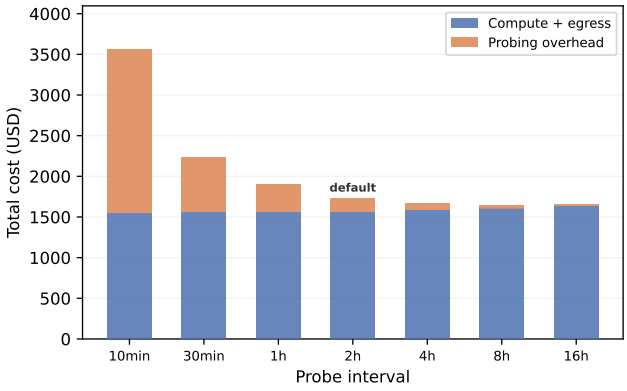


Figure 14: Cost breakdown vs. probe interval. Compute cost (blue) remains nearly constant (∼6% variation), while probing overhead (orange) dominates at short intervals. The default 2-hour interval balances information freshness against probing cost.

tion, applying a first-order Taylor expansion, and reducing to a per-action ranking.

Step 1: Effective progress rate and amortized cost rate. From state (t,p,r_0), action $a=(r,m)$ launches an instance with expected episode length $\bar{L}_{(r,m)}$, cold start d, compute rate $C_{(r,m)}(t)$, and one-time migration cost $E_{r_0\to r}$. Define the *effective progress rate*

$$g_a \;=\; \frac{(\bar{L}_{(r,m)}-d)^+}{\bar{L}_{(r,m)}} \;=\; \eta_{(r,m)},$$

the fraction of each episode spent doing useful work. By a standard renewal argument [8], the long-run average cost rate of repeatedly executing action a is

$$\ell_a \;=\; C_{(r,m)}(t) \;+\; \frac{E_{r_0\to r}}{\bar{L}_{(r,m)}},$$

where the second term amortizes the one-time migration fee over the expected episode length.

Step 2: Bellman equation and Taylor expansion. Over an infinitesimal interval $\mathrm{d}t$, the Bellman equation at (t,p,r_0) reads

$$\hat{J}(t,p,r_0) \;=\; \min_a\Big\{\,\ell_a\,\mathrm{d}t \;+\; \hat{J}(t+\mathrm{d}t,\; p+g_a\,\mathrm{d}t,\; r)\,\Big\}.$$

Our surrogate $\hat{J}$ (§3.3) depends on (t,p) and uses the reference on-demand price, so it is effectively region-independent: $\hat{J}(t,p,r)\approx\hat{J}(t,p,r_0)$. Expanding $\hat{J}$ to first order in $\mathrm{d}t$:

$$\hat{J}(t{+}\mathrm{d}t,\; p{+}g_a\mathrm{d}t) \;=\; \hat{J}(t,p) \;+\; \frac{\partial\hat{J}}{\partial t}\,\mathrm{d}t \;+\; \frac{\partial\hat{J}}{\partial p}\,g_a\,\mathrm{d}t \;+\; O(\mathrm{d}t^2).$$

Substituting back and cancelling $\hat{J}(t,p)$ from both sides:

$$0 \;=\; \min_a\Big\{\,\ell_a \;+\; \frac{\partial\hat{J}}{\partial t} \;+\; \frac{\partial\hat{J}}{\partial p}\,g_a\,\Big\}.$$

Step 3: Reduction to utility ranking. Since $\partial\hat{J}/\partial t$ is independent of a, the optimal action is

$$a^* \;=\; \arg\min_a\Big\{\,\ell_a+\frac{\partial\hat{J}}{\partial p}\,g_a\Big\} \;=\; \arg\max_a\Big\{\,\underbrace{-\frac{\partial\hat{J}}{\partial p}}_{=V}\,g_a \;-\; \ell_a\Big\}.$$

Substituting the definitions of V, g_a, and ℓ_a from Steps 1–2 yields

$$a^* = \arg\max_a\; U_{(r,m)},$$
$$U_{(r,m)} = V\cdot\eta_{(r,m)} - C_{(r,m)}(t) - \frac{E_{r_0\to r}}{\bar{L}_{(r,m)}},$$

which is eq. (6). The functional form of U is determined entirely by the Bellman structure; the choice of $\hat{J}$ affects only the scalar V.

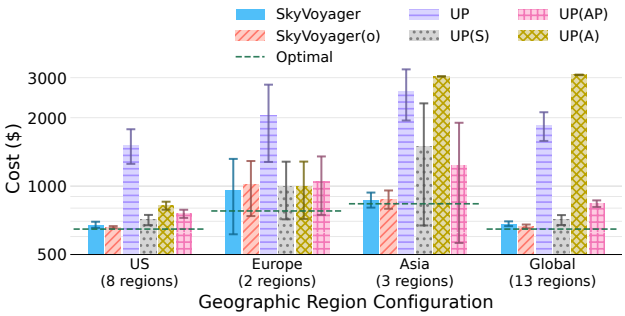


Figure 15: Cost under geographic constraints (US-only, Europe-only, Asia-only, and Global). `SkyVoyager` adapts effectively across all configurations, achieving significant savings even with limited regional diversity. Error bars show 95% confidence intervals over 20 jobs.

A.5 Sensitivity to Uniform Q-Error

This bound is mathematically correct but numerically vacuous in our setting because the dominant term compares a causal reservation price to a hindsight oracle costate. We retain it as a sensitivity result showing how approximation quality decomposes, not as a deployable guarantee.

Lemma 1 (Sensitivity to Uniform Q-Error)**.** *Let* $Q_t^{off}(s,a;\omega)$ *denote the offline one-step Q-value on a realized trace* ω*. If*

$$\sup_{s,a} \left|\hat{Q}_t(s,a) - Q_t^{off}(s,a;\omega)\right| \;\le\; \varepsilon_t$$

for all t *on trace* ω*, then*

$$\mathrm{Cost}(A,\omega) \;\le\; \mathrm{OPT}^{off}(\omega) + 2\sum_{t=0}^{T-1} \varepsilon_t.$$

With uniform $\varepsilon_t \le \varepsilon$*, this gives* $\mathrm{Cost}(A,\omega) \le \mathrm{OPT}^{off}(\omega) + 2T\varepsilon$.

The per-step error ε_t decomposes as

$$\varepsilon_t \;\le\; g_{\max}\cdot\left|\hat{V}_t - V_t^*\right| \;+\; \left|\hat{V}_t\right|\cdot\left|\hat{\eta}_t - \eta_t\right| \;+\; \left|\hat{C}_t - C_t\right| \;+\; \rho_t^{\mathrm{lin}},$$

where ρ_t^{lin} is the Taylor residual of $\hat{J}$ as an approximation of J^* over one tick.

A.6 Walkthrough of Migration Decisions in Figure 8

We provide three representative decisions from the L4 trace in Figure 8 to illustrate how `SkyVoyager` applies the unified cost model.

(1) $t_1 = 5.6$ **hr:** After preemption in `us-west-2c`, `SkyVoyager` evaluates all available regions. Probing shows `us-east-2c` ($1.81/hr) is available with predicted lifetime of 4 hours. At this point, $V(t) \approx$ $2.6/hr, and after accounting for cold start overhead (6 minutes), each hour of effective progress is worth $2.5. The $2 egress cost, amortized over the 4-hour lifetime, adds $0.50/hr. The combined per hour cost is $1.81 + $0.50 = $2.31/hr, which is lower than the monetary value of progress, yielding positive utility and `SkyVoyager` thus migrates.

(2) $t_2 = 13.0$ **hr:** After 4 preemptions in `us-east-2` (hours 9–13), the job has fallen behind schedule with deadline pressure $\theta(t) = 0.73$ (vs. nominal $P/T = 0.67$; see §3.3), raising $V(t)$ to $\approx$ $4/hr. At this elevated value of progress, even the more expensive `ap-northeast-1c` ($2.32/hr) yields positive utility. Meanwhile, `us-east-2` is detected to be in a volatile period ($\gamma^* > 1$), reducing its predicted lifetime to under 1 hour. `SkyVoyager` then selects `ap-northeast-1c` as the only region with positive utility and tolerates the $2 egress cost.

(3) $t_3 = 16.3$–20.0 **hr:** After a spot preemption in region `ap-northeast-1c`, `SkyVoyager` attempts to launch in the cheap `us-east-2` regions ($1.80/hr) first (highest utility), but they are unavailable. Other available regions like `us-west-2c` ($2.35/hr) appear volatile and yield near-zero or negative utility given $V(t) \approx$ $2.6/hr, while idling has $U = 0$. `SkyVoyager` waits, and as deadline pressure increases, $V(t)$ rises until `eu-central-1a` ($2.65/hr) finally yields positive utility at hour 20, at which point `SkyVoyager` launches there.

A.7 Progress Value Ablation

The progress value $V(t,p)$ is the only design knob in the utility (eq. (6)). We compare five alternative V candidates against our log-barrier default (eq. (8)) and the Optimal policy (§3.3) across four benchmark cells. All candidates use the same utility pipeline (§3.2) with perfect lifetime prediction, isolating V as the only variable.

Candidates.

- **Log-barrier** (default): $V = C_{\mathrm{od}}\,\theta/\tilde{\theta}$, the costate of the log-barrier surrogate (eq. (7)).
- α=0.5 (sublinear): $V = C_{\mathrm{od}}\,(\theta/\tilde{\theta})^{0.5}$, a power-family generalization of eq. (7) with a different urgency response.
- **Quadratic surrogate**: $V = C_{\mathrm{od}} \cdot \frac{t}{T-t} \cdot \frac{P-p}{P}$, the costate of a quadratic barrier $\hat{J} \propto (P-p)^2$.
- **HJB exponential**: $V = C_{\mathrm{od}} \cdot \exp(\theta/\tilde{\theta} - 1)$, exponential urgency anchored at $V = C_{\mathrm{od}}$ on schedule.
- **Time-only**: $V = C_{\mathrm{od}} \cdot t/(T-t)$, deadline urgency without progress awareness.
- **Neely DPP**: $V = C_{\mathrm{od}}\,(P-p)/P$, the Lyapunov drift-plus-penalty weight [41]; ignores time entirely.

Setup. We evaluate on four cells: V100 4-region and H100 8-region, each at deadline ratios $T/P \in \{1.5, 2.0\}$, with 12 stratified-sampled traces per cell. The V100 cells have a spot/on-demand price ratio of $\sim$3$\times$; the H100 cells have $\sim$5.6$\times$. Cost gap is the mean per-trace cost minus the Optimal cost.

	V100 4-region		H100 8-region		
V candidate	1.5×	2.0×	1.5×	2.0×	Σ
Optimal	$33.15	$24.76	$522.72	$390.93	—
Log-barrier (default)	**+9.60**	+5.83	+62.59	**+30.92**	**+108.94**
Quadratic surrogate	+30.95	**+5.11**	**+38.42**	+35.26	+109.74
α=0.5	+9.86	+5.54	+64.70	+31.26	+111.36
HJB exponential	+9.90	+5.50	+66.20	+33.91	+115.51
Time-only	+27.73	+8.52	+40.91	+44.26	+121.42
Neely DPP	+10.67	+9.98	+68.88	+35.67	+125.20

Table 2: Cost gap ($) vs. Optimal across all ablation cells. Lower is better. Bold marks per-cell best. Column headers show the deadline ratio T/P. All candidates use the same utility pipeline; only the V formula differs.

Results. The log-barrier default achieves the lowest total cost gap ($108.94), winning the V100 1.5× and H100 2.0× cells. The quadratic surrogate is a close second overall ($109.74) and wins on H100 1.5× and V100 2.0×, but is volatile: on V100 1.5× its cost gap ($30.95) is 3.2× higher than the log-barrier's, because its linear dependence on $(P-p)/P$ underweights urgency when progress is small early in the job.

Candidates that drop one of the two components of the schedule ratio perform worst. Neely DPP, which ignores time entirely, is worst overall ($125.20); Time-only, which ignores progress, is second-worst ($121.42). The full schedule ratio $\theta/\tilde{\theta}$ captures both deadline urgency and progress deficit, yielding the most consistent performance.

Summary. The log-barrier $V = C_{\texttt{od}} \cdot \theta/\tilde{\theta}$ achieves the lowest total cost gap across all four cells. No alternative matches its combination of low average gap and cross-cell consistency.